UBS VERIFICATIONS · DECISION ENGINE · JULY 2026

The Verifications Decision Engine

2.75 million replayed case-days — and the learning operation that evidence can unlock. Five pieces: the business case, the ceiling of today's automation, one case through the machine, the engineering, and the philosophy. Nothing sent without a person — yet: automation is a later, earned stage.

Reading notes. All figures are backtested and as-of (no future information); correlations are observational — no channel or wording is claimed to cause outcomes until the pilot measures it; the system is decision support today — nothing auto-sends yet; automation is designed in as a later, evidence-gated stage. The planned next layers: an AI contact researcher that finds fresh, verified contact paths, and a small language model that drafts the emails, faxes, notes, and call scripts — sequenced last, behind the causal instrumentation that can grade them.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

HOW TO READ THIS PACKET

The argument in five parts

1	**The executive briefing**	establishes the business problem, the working foundation, and the destination
2	**The Murphy Brown field report**	shows why scripted execution needs an operating layer around it
3	**The illustrated walkthrough**	follows one case through that layer — today, and in the next stage
4	**The engineering deep dive**	shows how the system is built, tested, and extended
5	**Earned autonomy**	explains why each increase in authority follows evidence from the stage before it

One sentence carries all five: six machine-learning models find the winnable work, the operating system makes acting on it safe, the pilot teaches it what works — and each completed case makes the next decision better.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

EXECUTIVE BRIEFING

The Verifications Decision Engine

The waste in verifications is countable now. One kind alone — re-contacting people who had already answered — burns roughly ten operators' worth of hours a year. And the number we manage by can't tell a case that was impossible from a case we lost. I replayed 2.75 million of our own case-days to count it, and built the fix in the same pass: a fleet of specialist machine-learning models that reads every open case each morning, inside an operating system that turns the scores into each case's next safe move. Already backtested, on our own cases. Next: wire it into the live workflow, Murphy Brown included — the hours come back, turnaround drops, the UTV number finally moves, and the record the pilot keeps starts teaching the engine which complete play wins.

The problem 1 in 5 cases ends unable to verify **The evidence** 2.75M real case-days replayed

Status built & backtested — next step: integration

THE 60-SECOND VERSION

- **What was built:** two halves that only work together — a fleet of specialist machine-learning models that reads every open case every morning and scores how it will end (right 71% of the time today, ~81% at day 0), and the operating system that turns those scores into safe action: hard rules that make an improper contact structurally impossible, an orchestrator that recommends each next move. Nothing auto-sends yet.
- **What it proved:** replayed against the full year of casework — a 48%-vs-3% difficulty spread the blended UTV (unable-to-verify) number hides, 1 case-day in 7 where we would have re-contacted someone who had already replied, and zero policy violations.
- **Next steps:** wire the engine into the live workflow — Murphy Brown included — with outcome logging on, adopt the achievable-UTV scoreboard, and let the logged evidence switch on the next layers in order: learn the winning play, repair dead contact paths, draft the outreach — and only then earn auto-send, lane by lane.

What was built: machine learning can read a caseload; it can't run one — so this is two things that only work together. The intelligence: a fleet of specialist models, trained on a year of our own case history, that reads every open case each morning and scores how it will end. And the operating system that makes intelligence safe to act on in a regulated department: a deterministic rulebook that decides what's allowed before any score is consulted, an orchestrator that turns scores into each case's next move, a flight recorder that logs every recommendation against what actually happened. **The models find the winnable work; the operating system makes acting on it safe.**

Start with the problem: **one in five worked cases ends "unable to verify,"** and for years the number could be watched but not managed. Behind each one is a person waiting — on a job offer, a lease, a placement; that wait is what turnaround time measures. Replaying a year of casework showed why — it isn't one number. Three different problems wear the same label:

- **Cases nobody could win.** The hardest tenth fails 48% of the time no matter what anyone does — yet today it gets the same effort as everything else.
- **Winnable cases lost to routine.** Uncapped call loops run 47–69% never-verify while queues are worked in arrival order and urgent, savable cases age.
- **People contacted after they already answered.** On 1 case-day in 7, the reply was on record and the flow would have reached out again anyway.

None of that is visible in a blended average — and all of it is fixable with the same machinery. The operation's asks were three: pick the next move on every case, know each case's fate upfront, and prove which outreach actually works. I built one capability per ask — the first two are built and backtested; the third is deliberately sequenced behind the pilot's outcome logging, and the roadmap below says exactly how it earns its way in. Everything below is the evidence. Four numbers carry it:

71%

How often it correctly spots the cases that will never verify

Rises to ~81% at day 0 — before any work has been done on the case.

48% vs 3%

Failure rate: hardest tenth vs easiest tenth of cases

Same teams, same process — a 16× spread the old metric couldn't see.

1 in 7

Case-days where the person had already replied

The current flow would contact them again. The engine catches it first.

0

Policy or do-not-contact violations

Across all 2.75 million historical case-days replayed. By design, not by luck.

Evidence base: roughly 880,000 real cases (June 2025 – May 2026), replayed as 2.75 million daily snapshots; 16.5 million candidate moves evaluated; every prediction checked against the outcome that actually followed.

How to read the scores — sixty seconds, no math

Every prediction score below works the same way. Imagine handing the system two cases — one that eventually verified, one that never did — without telling it which is which. **The score is how often it puts them in the right order.** 50% is a coin flip. 100% is a crystal ball. Around 70% is genuinely useful — roughly the sorting power a credit score has for lending risk, and banks run on those.

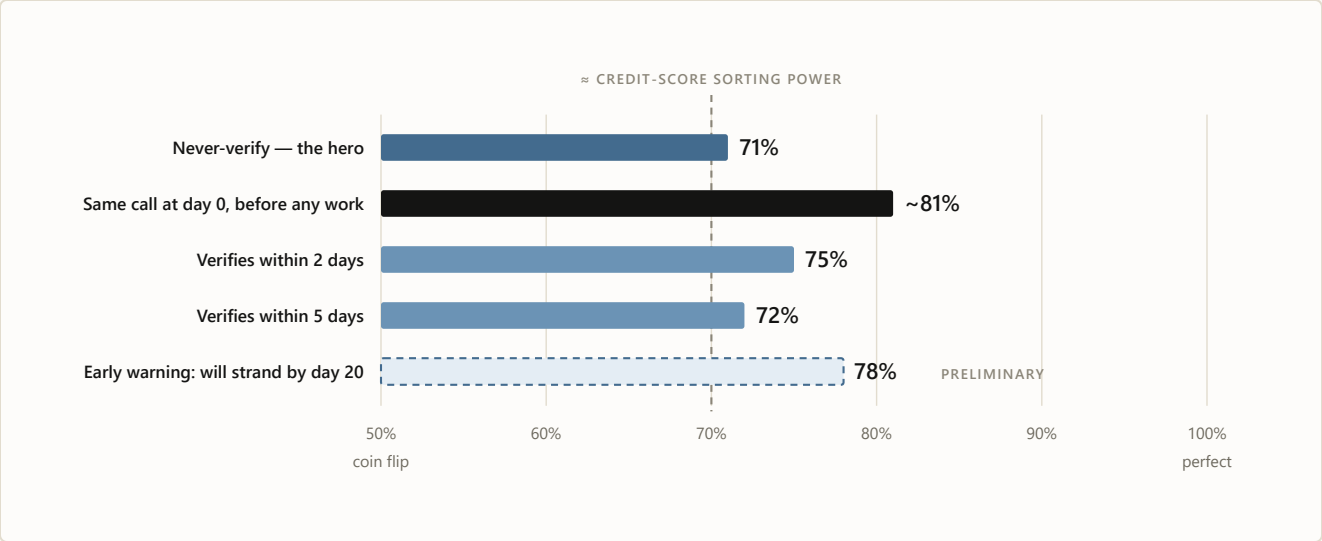


CHART 1 All scores measured by replaying history the honest way: for each day of each case, the models were shown only what was knowable that morning, then judged against what really happened. The two success models were upgraded this month — the improvement was re-proven on a month the models had never seen before promotion. "Preliminary" means exactly that: one month of evidence, still being validated, not yet load-bearing.

TAKEAWAY On the day a case arrives, the engine is already 62% of the way from a coin flip to perfect — before a single attempt is spent. Today's process has no read at all: every case enters the same queue at the same cadence.

Three things about that chart deserve a beat. The 81% is the fleet reading a case *at the moment it arrives* — nothing but intake facts — at its sharpest power of the day; that was the second ask, verbatim: *know upfront*. The pooled 71% is the harder everyday question — re-ranking the survivors after the easy cases resolve out — so the score falls as the question hardens, not as the

models age. Part of the accuracy comes from reading: 17.9 million free-text operator notes, distilled into signal by an 11-class classifier at export time with the raw text never stored — the first step toward a system that learns to read before it is allowed to write. And the build speed is evidence too: the first working version was a one-week build shown on June 19, and by the champion cycle two weeks later the five-day read had moved from 67% to 72% and the day-0 read from 73% to ~81% — each improvement forced through the promotion gate, not a demo bar. The founding question — how quickly can viable SLMs be produced on our own casework — already has its answer: weeks, in writing.

The spread is the strategy

Scores on individual cases are interesting. What changes how the operation is run is the population view: sort all cases by predicted difficulty and look at the two ends.

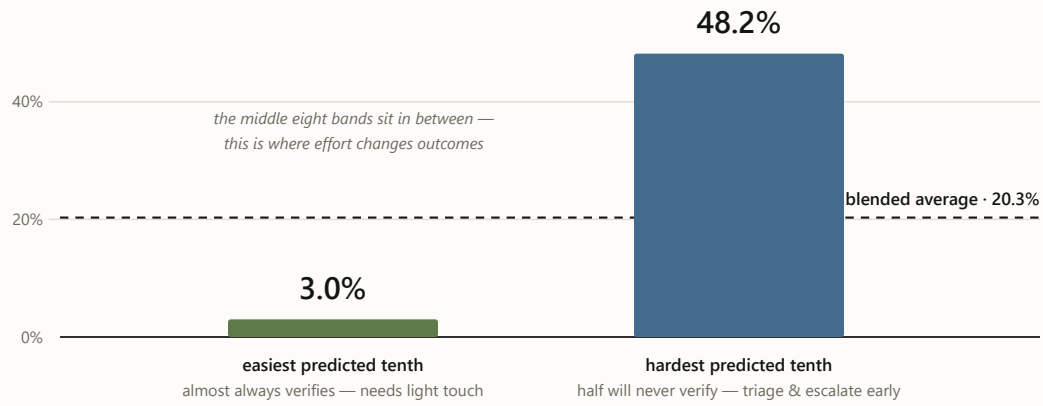


CHART 2 Actual never-verify rates by predicted band (May 2026 holdout). The blended 20% average hides a 16× spread. Once you can see it, "unable to verify" stops being one number and becomes a strategy: protect the easy, fight for the middle, stop over-investing in the structurally dead — and measure teams against what was genuinely winnable.

TAKEAWAY Today, one blended number treats a 3%-risk case and a 48%-risk case identically — a 16× spread, invisible. Ranked triage packs 2.4× more true never-verifies into the first list worked each morning than today's arrival-order queue: same people, same hours, more saves.

The model doesn't just predict the future. It makes the UTV number fair — misses measured against what was actually possible.

How intelligence becomes a safe next move

Every morning the model fleet — six specialists — reads the same case snapshot, a 147-signal view of everything knowable that day, and returns calibrated probabilities: when it says 30%, it happens about 30% of the time. It does not send, block, close, or authorize anything. From there the operating system takes over: the resolver names the case state, deterministic rules remove unsafe moves, the scorer ranks what remains, a person decides, and the flight recorder connects the recommendation to what actually happened. No single component runs the case — the handoff between them does:

Machine learning decides what deserves attention. The operating system decides what may happen. A person decides what happens now.

1 · The model fleet reads & predicts	Reads the morning snapshot and forecasts the case's path — the specialists: never-verify, two-day, five-day, the day-0 intake read, the re-scores, the early warning.
2 · The case-state resolver names what is true	Thirteen states — replied, blocked, cooling down, workable — decided by rules, never by a score.

3 · Deterministic rules remove unsafe moves	Do-not-contact, client policy, timing, "they already replied" — a blocked move leaves the table before ranking begins.
4 · The ML-backed scorer ranks what remains	Only surviving moves get ranked; no confidence level resurrects a removed one.
5 · A person decides	Takes the recommendation or overrides it — and the override is as valuable a signal as the action.
6 · The outcome record teaches	Connects each recommendation to what actually happened; that complete chain retrains the fleet monthly, behind the promotion gate.

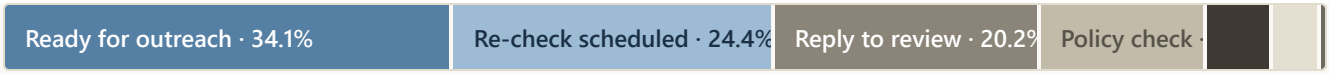
Pressure-test the handoff: suppose a specialist strongly favors another call on a case whose reply is already on record. The rules remove call before ranking — no confidence level restores it, and the person never even sees it as a recommendation. That is why the replay's violation count is zero: the models never get the chance to cause one.

One more reason the scores can be trusted: every test scored each case using **only what was known that specific morning** — "as-of" discipline, enforced by an automated no-peeking audit that re-checks every new training matrix before the models may train on it. The scores here are conservative by design, measured the hard, honest way.

The same discipline applies to upgrades: a new model version replaces the old one only if it wins on every criterion — including on a month of data it has never seen. That gate has rejected three of my own "improvements" this year because they would have weakened the top of the worklist, the part your teams actually touch. The system says no to me in writing. That's a feature.

What the operation gets on day one

Every case-day is sorted into a named state with a next step — nothing waits silently in a queue with no reason attached:



- Ready for outreach — ranked move attached 34.1%
- Scheduled re-check, with a reason 24.4%
- A reply is waiting — review before contacting 20.2%
- Policy / do-not-contact needs human judgment 12.7%
- Contact prohibited — fully blocked 4.9%
- Cooling down after a recent attempt 3.7%
- Escalation lane — normal paths exhausted 0.2%

CHART 3 Where all 2.75M case-days actually sit. A third of the book is workable *today* with a ranked move attached; a fifth has a reply sitting unread by the current flow. This chart is, in effect, tomorrow morning's org-wide to-do list.

TAKEAWAY This works on day one, no pilot required: every case-day carries a named state, a reason, and a next step — and the 1-in-7 accidental re-contact becomes structurally impossible instead of vigilance-dependent.

For an operator, the difference is the first five minutes of the morning (illustrative): the queue opens already ranked, the top case says why it is first, and the reply that arrived overnight is flagged for review — before it can become an accidental re-contact.

That is also where the turnaround-time story lives. Velocity doesn't come from asking people to work faster; it comes from deleting dead motion and moving recoverable cases earlier — replies go to review before another touch is spent, handbacks re-enter ranked by urgency instead of waiting behind fresh work, and stalled cases switch channel or escalate before another day is lost.

Where the hours come back

THE IMPACT ARITHMETIC — MEASURED COUNT, ILLUSTRATIVE CLOCK

The replay measured roughly **400,000 case-days a year** where the flow would have re-contacted someone whose reply was already on record. That count is measured, not assumed. Put an illustrative three operator-minutes on each avoided touch and it is **~20,000 hours — roughly ten operator-years** — before a single capped dead-lane call or faster handback is counted. The three-minute clock is the only assumption in the line; the pilot's logs replace it with measured handle time, and that number becomes the business case. I deliberately quote no labor-savings figure beyond this arithmetic — that figure belongs to the pilot.

What the department looks like after implementation

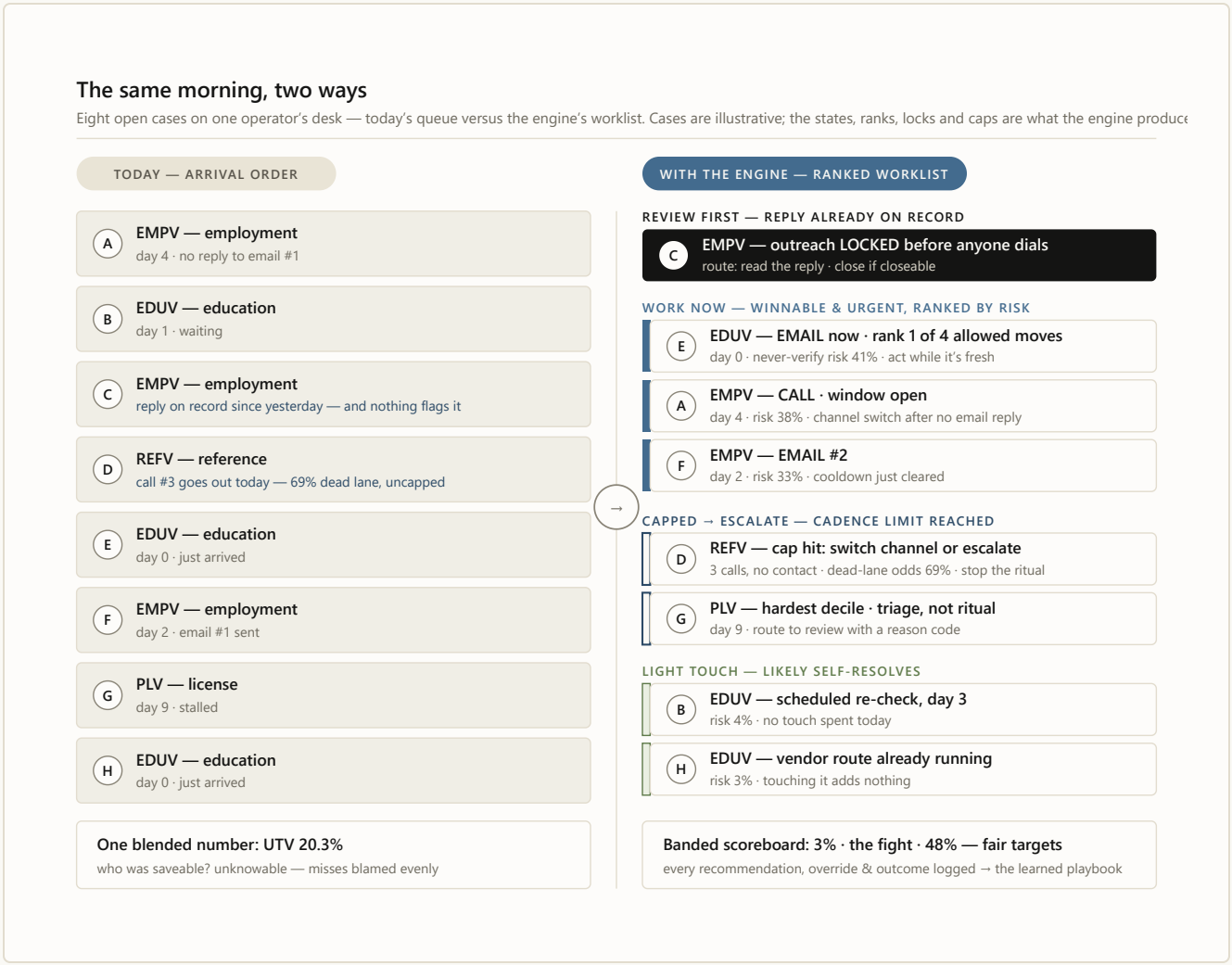


CHART 4 The same eight cases, two mornings. Case details are illustrative; the locks, caps, ranks and states are what the backtested engine produces on real case-days — states from the deterministic resolver, order from the calibrated risk scores. On the left, C and D are problems nobody can see. On the right, they are named states with owners, and the operator's first hour goes where it can still change the outcome.

TAKEAWAY Nothing about the cases changed — only the reading did. C stops being tomorrow's accidental re-contact, D stops burning a third call into a 69% dead lane, and the first hour of the morning goes to E, A and F, where effort still changes outcomes.

We already know what automation without this layer looks like

This isn't hypothetical. Our current verification bot — Murphy Brown, run by SNH-AI — automates the motion, not the mission: over four months, **55.5% of the tens of thousands of cases it touched flowed back to people** as note-only handoffs, its closures now reopen at roughly ten times the winter rate, and its education handbacks wait **+30.6 hours** behind fresh work. The deep-dive's own fix list — structured handoff state, closure lock, score-gated pickup, handback re-ranking, outcome logging — is, item for item, the system in this briefing; inside it, the bot becomes a governed channel whose speed finally serves a mission. The full story, held to its own strict claim discipline, is in the Murphy Brown field report.

From decision support to a learning operation

The first release recommends; the destination learns. **Now — read and recommend:** the model fleet scores every open case, deterministic rules strike the unsafe moves, the engine recommends, a person decides. **Next — learn the complete play:** the live workflow logs recommendation, action or override, channel, timing, message, delivery, reply, and outcome — the record that teaches which complete play wins each kind of case. **Then — repair the inputs and draft the outreach:** the contact researcher hunts fresh, verified emails, fax numbers, and phone numbers, so repairing the input replaces re-keying around it — the standing ask on manual entry starts here — and the SLM drafts the emails, faxes, notes, and call scripts from what the log proves works. **Finally — automate what earns it:** a narrow lane moves to controlled auto-send only after its logged evidence proves it safe, stable, and effective; higher-risk and uncertain cases stay with people. The long-term product is not a model or a bot — it is a verification operation that improves after every completed case.

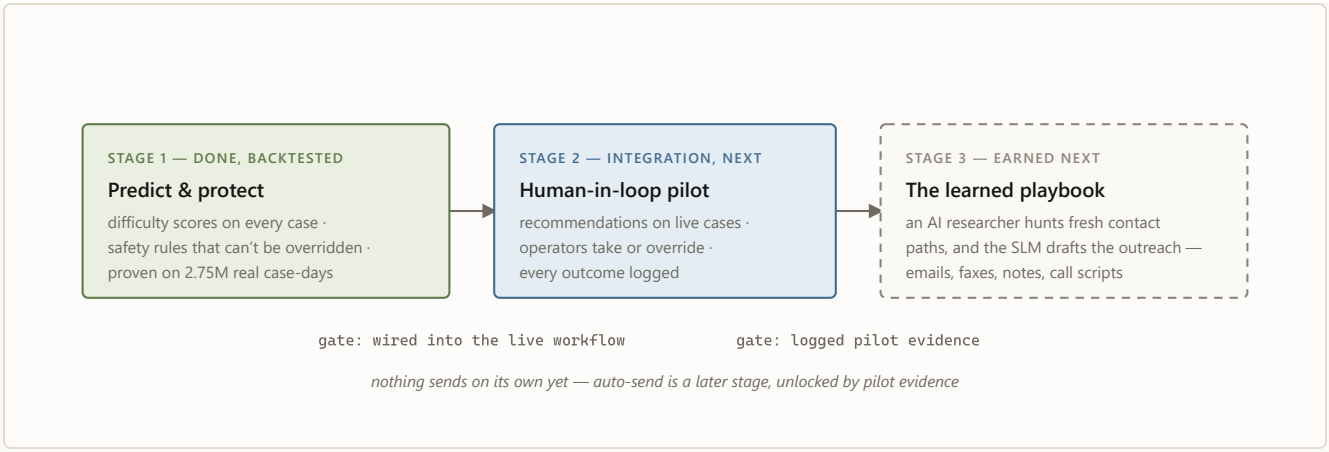


CHART 5 The roadmap. Stage 1 is finished and measured. Stage 2 is the integration step — the engine running beside the people who work the queue. Stage 3 — the part every vendor promises on day one — is deliberately last, because it gets proven rather than claimed.

Two questions worth asking

Q · "So an AI decides who gets contacted?"

No. The AI only *orders* options that a deterministic rulebook has already approved — and today a person executes. It cannot add an option or skip the rulebook, and it does not send anything on its own — *yet*. The plumbing for automation is already built; switching it on is a later stage that pilot evidence unlocks, lane by lane, starting with the safest.

Q · "What happens when the model is wrong?"

Someone works a less-urgent case first. That is the entire blast radius — model error can cost efficiency, never compliance, because what's allowed is never the model's call. The scores are also calibrated: when the system says 30%, it happens about 30% of the time, so error is budgeted rather than hidden.

The decision, and the gates on it

- **Adopt the fair "achievable UTV" scoreboard** and track it against a three-month forward test — turning Chart 2 into the operation's scoreboard.

- **Integrate the engine into the live workflow with outcome logging on** — recommendations beside the people who work the queue, Murphy Brown as a governed channel inside it, every recommendation, override (with its reason), send, reply, and outcome recorded. The gates ride along: overrides are measured instead of assumed away, monthly retraining behind the promotion gate counters drift, and logging ships with the integration so the pilot cannot come back unprovable. Two bot-side asks — the structured payload and reopen-reason codes — remain SNH-AI's to build; the engine works without them, but the bot stays a black-box channel until they land.
- **Let the next layers earn authority from the logged evidence, in order:** best-channel learning; the contact researcher repairing dead contact paths; the SLM drafting the outreach; then controlled auto-send, lane by lane, on each lane's own record. That evidence is ours alone — our department, our cases — a data advantage no outside vendor can bring or buy.

THE VERSION TO TAKE OUT OF THE ROOM

"Unable to verify" turned out to be three problems wearing one number. The models find the winnable work; the operating system makes acting on it safe — a fleet of specialists reading every case each morning, a deterministic layer deciding what's allowed before any score is consulted. Proven against 2.75 million of our own case-days, zero policy violations. The first release recommends; the live record teaches. And the honest, replay-tested scores are why you can believe the rest. ◆

The fine print, kept honest. All results are backtested (historical replay, as-of); no live-pilot results are claimed. Score "71%" = ROC-AUC 0.706, "~81%" = day-0 walk-forward check (0.80–0.82 across three forward months), "75%/72%" = this month's upgraded success models (0.747/0.717, re-proven on never-seen June data), "78%" = early-warning model, preliminary. Historical channel patterns are observational — no channel or message is claimed to *cause* outcomes until the pilot measures it. Action-facing models strip direct agent identifiers. Decision support today: nothing auto-sends yet and no decision is finalized by a machine — automation is designed in as a later, evidence-gated stage.

Deeper reading: the Murphy Brown field report (the problem this system closes) · the illustrated walkthrough (follow one case through the machine) · the engineering deep dive · the design philosophy.

A Wright Original · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · Executive briefing · July 2026

A Wright Original · Built by **SNH Strategic Initiatives**

FIELD REPORT

The ceiling of scripted automation

Murphy Brown, our verification bot, is genuinely fast when the contact path is already clear — it fully contained about 44.5% of the tens of thousands of cases it touched over four months. The other half flowed back to people with nothing but a note, and its closures are reopening at ten times the winter rate. What the ops deep-dive actually found, why the answer is to govern the bot rather than retire it — and how it becomes the execution layer inside the decision engine.

Evidence window Mar 1 – Jun 30, 2026 (live warehouse refresh)

Claim posture association language only — see the fine print

THE 60-SECOND VERSION

- **What works:** the bot fully contains ~44.5% of what it touches, usually within a day or two. Where the contact path is already laid and the case fits the script, it executes fast — genuinely.
- **What breaks:** the other 55.5% flows back to people as note-only handoffs; the closures it does make are reopening at ten-plus times the winter rate; and its handbacks queue behind fresh work (EDUV +30.6h).
- **Why it happens:** nothing gates what it picks up, nothing owns the state it leaves behind, nothing locks what it closes, and nothing records enough to prove any of it — so humans became the control layer. Not bugs: the ceiling of scripted automation.
- **What to do:** keep the bot; add the operating layer. Five asks — closure lock, triage gate, handback re-rank, one holdout config rule (the causal number lands ~July 15 either way), and the structured payload. The decision engine is that layer; the bot becomes a governed channel inside it.

Murphy Brown (“MB”) is the SNH-AI automation that works employment and education verifications alongside our people. Its measured strength is execution: where a contact plan is already laid, it fires the touch fast and often contains the case. Its measured limit is everything around the touch: a scripted bot cannot judge which cases are worth picking up, choose among contact paths, decide when to wait, switch, or escalate, or notice that a case it closed didn’t stay closed. That is the ceiling of scripted automation — and every number in this report is that ceiling, measured.

So the bot’s strongest result and its biggest problem turn out to be the same behavior: fast when the path is clear, judgment handed back when it is not. This is not an argument against Murphy Brown, and it is not an argument for turning it off. It is the case for governing it — adding the missing layer, the brain, that decides what should happen before and after every touch. The decision engine supplies the judgment; the bot supplies the speed.

Five numbers tell the story:

55.5%

Of bot-touched cases flow back to people

Roughly 14,800 of 26,700 — about four downstream human attempts apiece, 64,000+ rows in all.

~43%

Of the bot's closures now reopen after the result is sent

Up from ~3–4% in February. June figures are floors, not finals.

+30.6h

Extra wait for an EDUV handback vs. fresh work

Not a parking defect — 99.98% are agent-assigned. A queue-ranking problem.

1.7%

Containment in the worst tenth of what it picks up

Its wins were predictable before pickup (77% sorting power) — nobody was gating the door.

~45%

Of eligible tickets picked up by neither automation

About 113,000 EMPV/EDUV tickets where automation is live — the gate fails in both directions.

Source: the consolidated MB/SNH-AI findings (2026-06-30) and the July 2–3 forensics one-pager, read-only warehouse analysis. All figures are the current numbers of record for their stated windows.

A two-speed workflow

When MB touches a case, one of two things happens. About 44.5% of the time it fully contains the case — closed in one or two days, no human ever needed, genuinely fast. The other 55.5% of the time the case flows back to a person. That split, not any single defect, is the operating reality:

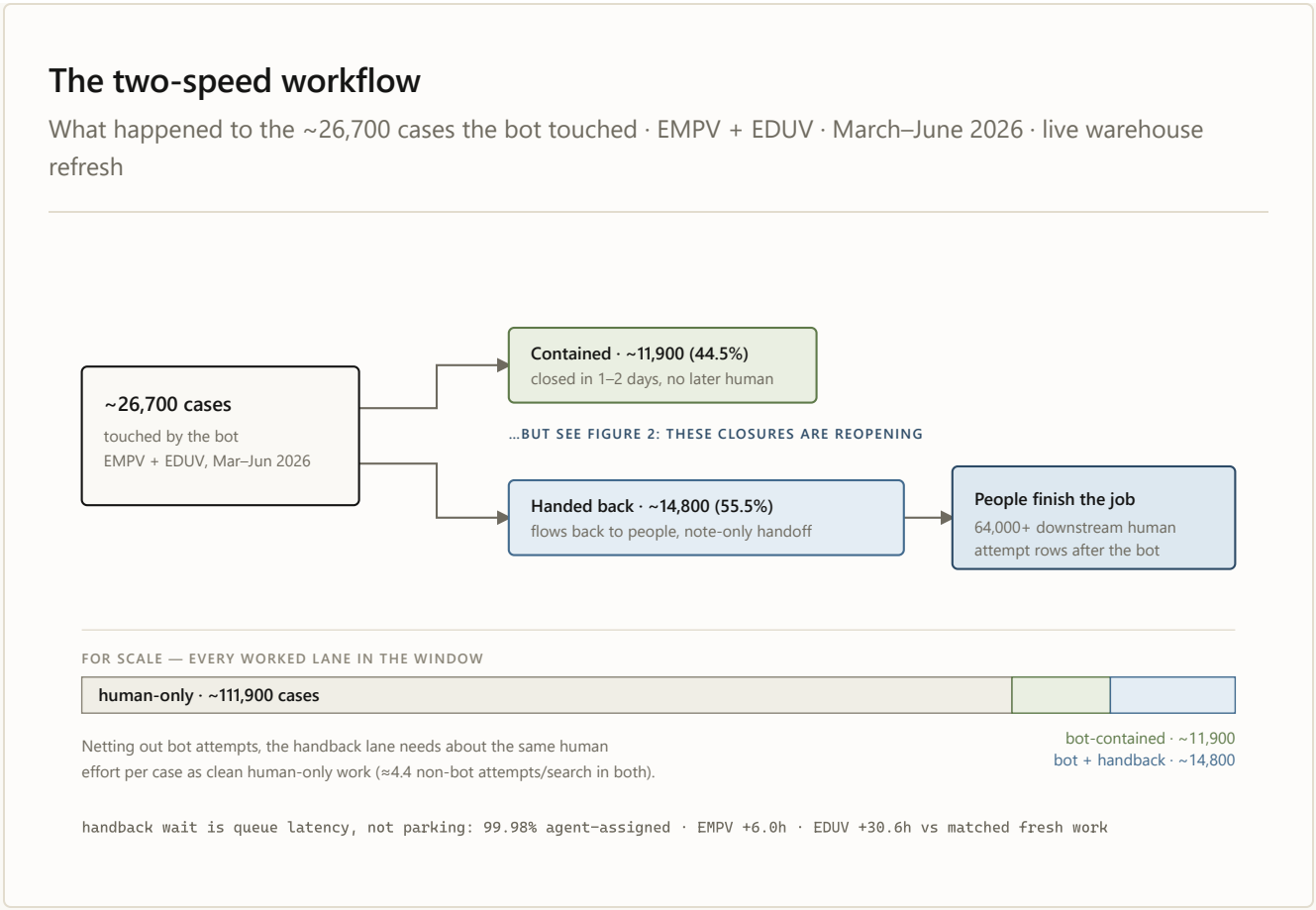


FIGURE 1 The two-speed workflow. The matched comparison is honest here: handback cases don't take clearly *more* human effort than comparable human-only cases — the durable finding is that the bot leaves a large residual queue with weak fast-closeout performance and no machine-readable handoff, so people must rediscover each case before continuing it.

Fairness requires the other half: **when the bot contains a case, it is genuinely fast.** The contained lane completes in about 61 hours versus 161 for human-only work, with a higher raw success rate (80.5% vs 73.7%). Two caveats keep that honest: these are raw lane averages, not matched comparisons — the bot picks up easier-looking work — and the clocks flatter it, because the bot fires instantly while human work queues first. Re-anchored at first touch, EMPV's speed advantage halves to −37.6 hours and EDUV flips to 23.7 hours *slower* than fresh human work. The containment is still real — which is exactly why the recommendation is to govern the bot, not retire it.

And there is a third speed the two-lane picture can't show: **never picked up at all.** Where automation is live, 44.9% of eligible EMPV/EDUV tickets — about 113,000 — were touched by neither the bot nor the SNH-AI engine behind it (measured per ticket). Part of the mechanism is already known: all-or-nothing API routing pushed about 50,000 excluded-client searches back to

manual work, a modeled 7.2–8.6k added manual searches per month that still needs configuration history to be proof-grade. The gate fails in both directions — the bot picks up cases it can't contain (its worst decile contains 1.7%) and never reaches nearly half of what it could. One gating layer fixes both.

And the contained lane — the bot's success story — has a quality problem growing inside it:

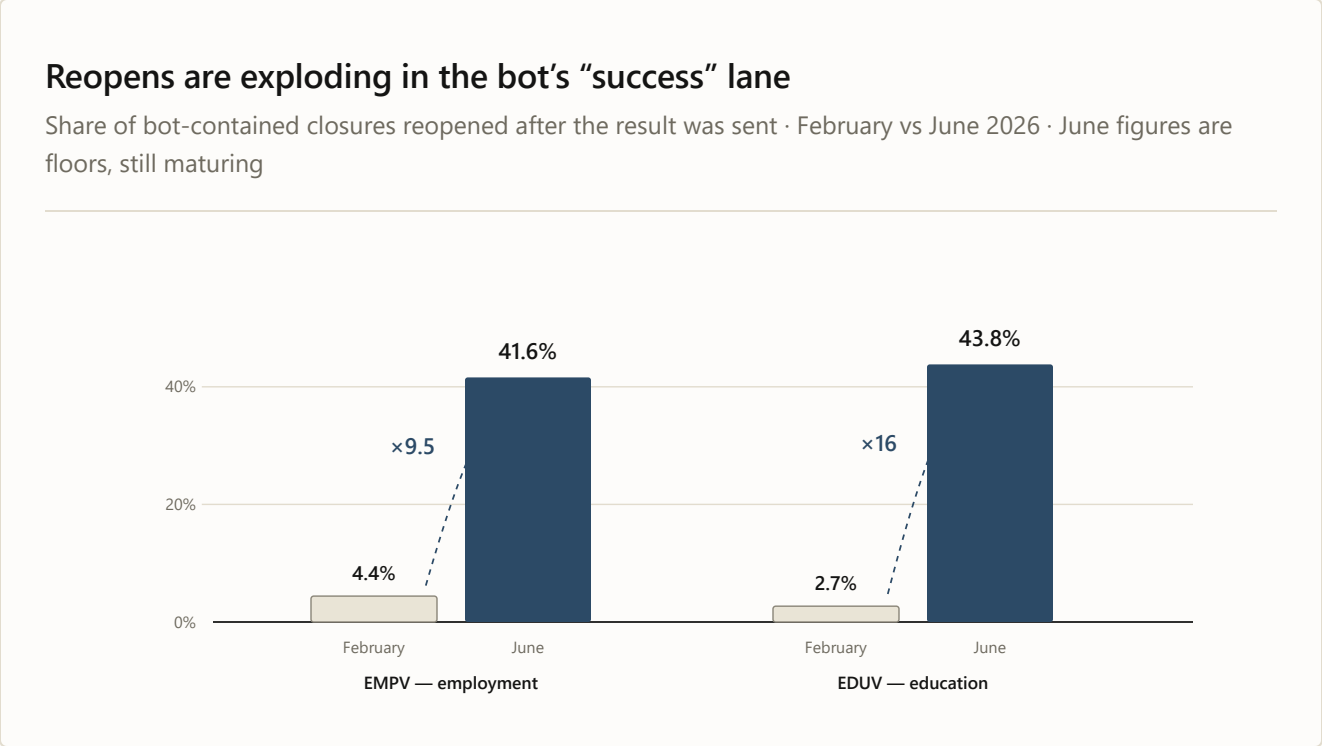


FIGURE 2 Reopen-after-result-sent on bot-contained closures, February vs. June 2026. June postdates are still maturing, so these are floors. The rise is within-lane and concentrated in bot-only closures — which is why the deep-dive's first ask is a closure lock with reopen-reason instrumentation, and why "grow containment" without one would scale a quality problem. The narrow June eForm defect (300 strict cases where the bot emailed *after* a closeout state, 1,119 manual touch rows) is the same missing lock in miniature.

A note is not a handoff

Underneath the volumes sits one mechanism, and the deep-dive names it precisely: *every strict MB touch in the diagnostic packet is note-only*. The bot does work and leaves an annotation — not a state. So the person who picks the case up must reconstruct what happened, whether evidence is complete, and what should happen next. The handoff itself creates work:

The handoff, as-is versus required

The deep-dive's core finding on the left; its required fix — the state contract — on the right

WHAT THE BOT LEAVES TODAY

"Emailed the employer contact. Will follow up."

note only · no outcome state · no next action · no owner · no retry date

WHAT A HANDOFF MUST CARRY (THE STATE CONTRACT)

`outcome_state`	completed · failed · waiting · needs-human · policy-blocked
`evidence_state`	is the case closeable right now?
`selected_contact`	which path it used — no rediscovery
`confidence`	route low-confidence work differently
`next_action`	prevents the "what now?" pickup
`owner`	who holds the case after the touch
`retry_due / cooldown`	stops same-day duplicate outreach
`pickup_reason`	separates valid continuation from rediscovery

FIGURE 3 The note versus the payload. The left side is the deep-dive's finding; the right side is its required fix — and it is, field for field, the shape of the decision engine's evidence-state and logging contracts.

The cost of that missing contract is now measured, not inferred: in the March–May window, humans **re-tried the exact channel the bot had already tried without success** — same method, same case — on roughly 6,700 cases, averaging 2–3 re-tries each (about 16,100 attempt rows), at a median five to six days after the bot's attempt. Scaled to the whole department, that is **roughly 3% of every named human attempt in the window — one touch in every thirty** — spent re-burning a channel the bot had already tried. (Method-level; per-target linkage is exactly what the payload's `selected_contact` field exists to provide.) A handoff that carried "what I tried" would have made each of those a choice instead of a rediscovery.

The rest of the issue list follows from the same missing layer — no judgment at the door, no memory behind it:

- **Thin contact data — 57.6% bounce back.** When a case arrives with weak source or contact information, the bot touches it anyway. Nothing checks whether the case was workable before pickup.
- **Fax-heavy and multi-path routes — 80.8% bounce back.** Wherever the path requires choosing between channels, the bot can't make the choice — so a person inherits the case right after the touch.
- **CALL-first retry loops — 35% end unable-to-verify; only 2% succeed within two days.** Repeat dialing with no stop rule, no channel switch, no escalation: the highest-friction path in the department, run on autopilot.
- **Policy exceptions — indistinguishable from avoidable work.** The rule that makes a wait legitimate was never encoded as data, so legitimate exceptions and avoidable rework look identical in the record.

Step back and one diagnosis explains every number in this report: **MB is execution without cognition.** Hand it a case with a clean, pre-laid contact path and it performs the motion — send the email, fire the fax — and does it fast. But a verification is rarely one motion; it is a sequence of judgments — which path first, when to wait, when to switch channels, when to escalate, when to stop, when a close is safe to call closed. The bot makes none of them. So it never picks up nearly half of what it could (no judgment about what is workable), hands back most of what it does pick up with only a note (no next move), leaves humans re-dialing channels it already burned (no memory), loops on calls without a stop rule (no cadence), and closes cases that will not stay closed (no closure judgment). No supervisor over its choices, no dispatcher behind its next move, no recorder under its work. The hands are fine. There is no brain.

Murphy Brown automates the motion, not the mission. It can send the email; it cannot run the case.

Why this is the decision engine’s story to finish

The decision engine supplies the operating layer the bot is missing: machine-learning models that read every case, a deterministic resolver that knows what is true and what is allowed, a scorer that picks the play, cadence rules that run the sequence to completion, and a log that remembers what was tried. That is not an analogy — the deep-dive’s fix list and the engine’s component list are the same list, item for item:

Their fix list is our component list

Each measured gap from the deep-dive, mapped to the decision-engine component that closes it

THE DEEP-DIVE FOUND		THE ENGINE HAS	STATUS
Note-only handoffs 55.5% flow back; humans rediscover each case	→	**Evidence-state resolver + decision contract** `13 states · movement card · owner, reason, next move on every case-day`	BUILT · BACKTESTED
Reopen explosion / eForm collision ~43% reopen; bot emails after closeout states	→	**Closure lock: terminal states + inbound-first rule** `resolved cases stop generating work · already-responded catch (1 in 7)`	BUILT · BACKTESTED
No triage gate at pickup worst decile contains 1.7%; wins predictable at 77%	→	**Difficulty models gate the door** `day-0 never-verify ~81% · score-gated eligibility routing`	BUILT · BACKTESTED
EDUV handbacks wait +30.6h queue-ranking, not parking	→	**Calibrated priority queue** `handbacks ranked by risk + urgency · review-due states carry triggers`	PILOT PROVES TAT
CALL loops, fax complexity, stale contacts repeat-call lanes at 35% UTV, 2% 48h success	→	**Cadence + eligibility controls** `escalation ladder · wait policy · eligibility rules — versioned configs`	BUILT · BACKTESTED

FIGURE 4 The deep-dive's asks mapped to the decision engine. Two items on the bot's side remain genuine asks to SNH-AI — the structured action payload and reopen-reason codes — because the engine can define the contract but the bot must write into it.

So the proposal is not "turn Murphy Brown off." It is: **Murphy Brown becomes a channel inside the decision engine.** The resolver decides *when* the bot may touch a case; the difficulty scores decide *whether it should* (fax-primary and thin-contact cases skip it — the deep-dive's own modeling says gating to the top ~60–70% keeps 83–91% of the bot's wins while halving the handback lane); terminal states and the inbound-first rule *lock what's closed*; the priority queue

re-ranks what comes back; and the flight recorder finally makes the bot's value a measured number instead of a debate.

What I'm careful not to claim

- **Association, not causation.** The bot is *associated with* a large downstream queue. The matched comparison does not prove it makes comparable cases worse on effort or time — the proven finding is the residual queue, the weak fast-closeout, and the missing handoff state.
- **No savings claims.** The handback lane's labor premium is small (≈$3/search). The money case is turnaround time and closure quality — 100,000+ EDUV wait-hours in the window, and reopen churn now measured at 0.69–3.4 reopens per automated closure (the leak-back band) but still unpriced in dollars. Attempt rows are not labor minutes, FTEs, or ROI.
- **No avoidable-percentage.** No share of the bot's downstream work is labeled "avoidable" — the record doesn't yet carry enough per-case evidence to make that judgment stick, so no avoidability number is quoted. The causal number comes from a two-week 50/50 holdout — one configuration rule on the bot's side, fully reversible — with a before-vs-after comparison against untouched cases running ~July 15 regardless.
- **Modeled is labeled modeled.** The missed-pickup mechanism (all-or-nothing API routing; about 50,000 exposed searches, ~7.2–8.6k modeled manual searches/month) still needs effective-dated configuration history to be proof-grade; the 44.9% neither-automation figure is measured per ticket where automation is live.
- **And one test the bot passed.** I probed whether MB fires before evidence exists: 0.0% of its touches happen while a vendor clock is open, and only 0.2–1.7% land before a vendor return. The bot's timing discipline is not the problem — its handoffs, closures, and gating are.

Five changes that make the bot governable

1. **Closure lock + reopen-reason instrumentation** — adjudicates the reopen explosion before anyone scales containment.

   `owner: SNH-AI · product / state-machine change`

2. **Triage gate at pickup** — score- or rule-based eligibility, re-tuned on a fresh window before any config change.

   `owner: joint · our scores, their routing`

3. **EDUV handback re-rank / SLA** — a worklist rule; kills the +30.6h penalty with zero bot work.

   `owner: UBS ops · smallest ask on the list`

4. **The holdout config rule** — two weeks, reversible, powered; the only path to the causal dollar-and-TAT number.

   `owner: SNH-AI · one configuration rule, fully reversible`

5. **Structured action payload + inbound capture** — the state contract of Figure 3, written by the bot, read by the engine.

   `owner: SNH-AI · the field spec is ready to hand over`

What the bot becomes

The same bot, two operating realities

What changes for Murphy Brown when the decision engine runs the case around it

MURPHY BROWN TODAY	MURPHY BROWN INSIDE THE ENGINE
Picks up whatever has a pre-laid contact path	The ML models pick the cases the bot is suited to win
Executes one scripted touch	Deterministic rules confirm the touch is allowed first
Leaves a note behind	Writes structured state and a next action
Returns uncertain work to the back of the queue	Handbacks re-enter ranked by risk and urgency
Cannot learn from the outcome	Every outcome becomes evidence for the next play

FIGURE 5 The same five behaviors, before and after governance. The left column is the measured present; the right column is the decision engine's existing components applied to the bot.

Murphy Brown is not the old system we discard — it is the execution layer the new system can finally use well. The decision engine decides which cases are ready, which actions are allowed, and what should happen next. As the pilot's log accumulates, the machine-learning models learn the complete play — channel, timing, message; an AI researcher finds a better contact path when the one on file is dead; a small language model drafts the outreach for a person to approve. Murphy Brown executes the proven plays at machine speed, lane by lane, as the logged evidence earns it.

Machine learning chooses the fit. Rules guard the touch. AI repairs weak paths and drafts the outreach. A person approves. Murphy Brown executes. The outcome improves the next case.

THE VERSION TO TAKE OUT OF THE ROOM

Today, Murphy Brown automates individual touches. Inside the decision engine, it becomes part of an operation that can run the entire case — and improve after every outcome. The engine runs the case; the bot becomes its fastest pair of hands — which makes Murphy Brown, honestly told, the strongest argument we have for shipping the engine. ◆

The series. This field report is the companion piece. The system it argues for: the executive briefing · the engineering deep dive · the design philosophy · the illustrated walkthrough.

Sources of record in the ops deep-dive workspace: `MB_MURPHY_BROWN_ALL_FINDINGS_CONSOLIDATED_20260630.md` and `outputs/exec_packet_20260702/MB_ONE_PAGER_20260702.md` (which supersedes earlier summaries; retired numbers per its `RETIRED_NUMBERS.md` are not cited here). All analysis read-only; windows as stated per figure; no client names, personnel names, or case identifiers appear in this report.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · Field report · July 2026

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

Meet **case 4127**: an employment verification, opened on a Monday. There is a work email and a phone number on file, no fax. Somewhere, an HR inbox is about to be asked to confirm a job history. Over the next nine days this case will be emailed twice, called once, and finally verified — and at every step, the system has to answer the same two questions: *how is this case doing?* and *what, if anything, should we do next?*

Case 4127 will not move through one model. It moves through a sequence of small decisions, each allowed to answer only one question — five components

taking turns. Here is the cast, and the color code every diagram in this post uses:

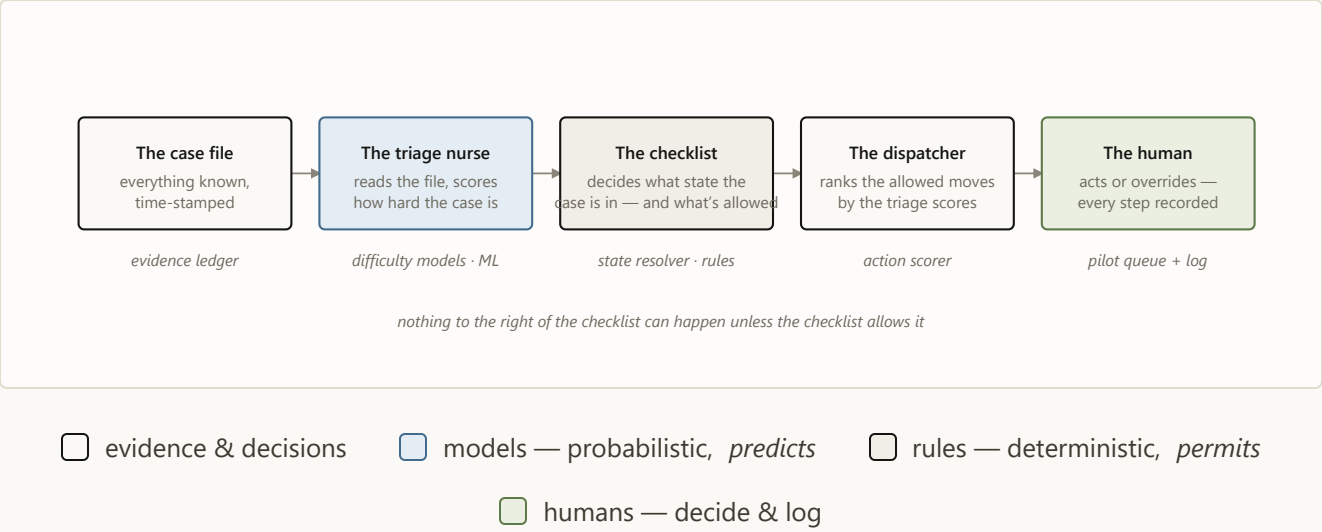


FIGURE 1 The cast, in the order they speak. The color code holds for every diagram below: blue predicts, oat permits, moss decides.

1 The curtain: what the system is allowed to know

The single most important idea in the architecture is not a model — it's a rule about time. The system never looks at a case as one blob of history. It looks at the case *as of a specific morning*, and it may only use facts from **before** that morning. One case therefore becomes many training rows — one per day it stayed open — and our twelve months of history becomes 2.75 million of these `(case, day)` snapshots.

Drag the day below and watch what case 4127 looks like from inside the system. Everything right of the curtain hasn't happened yet — as far as the machine knows, it never will.

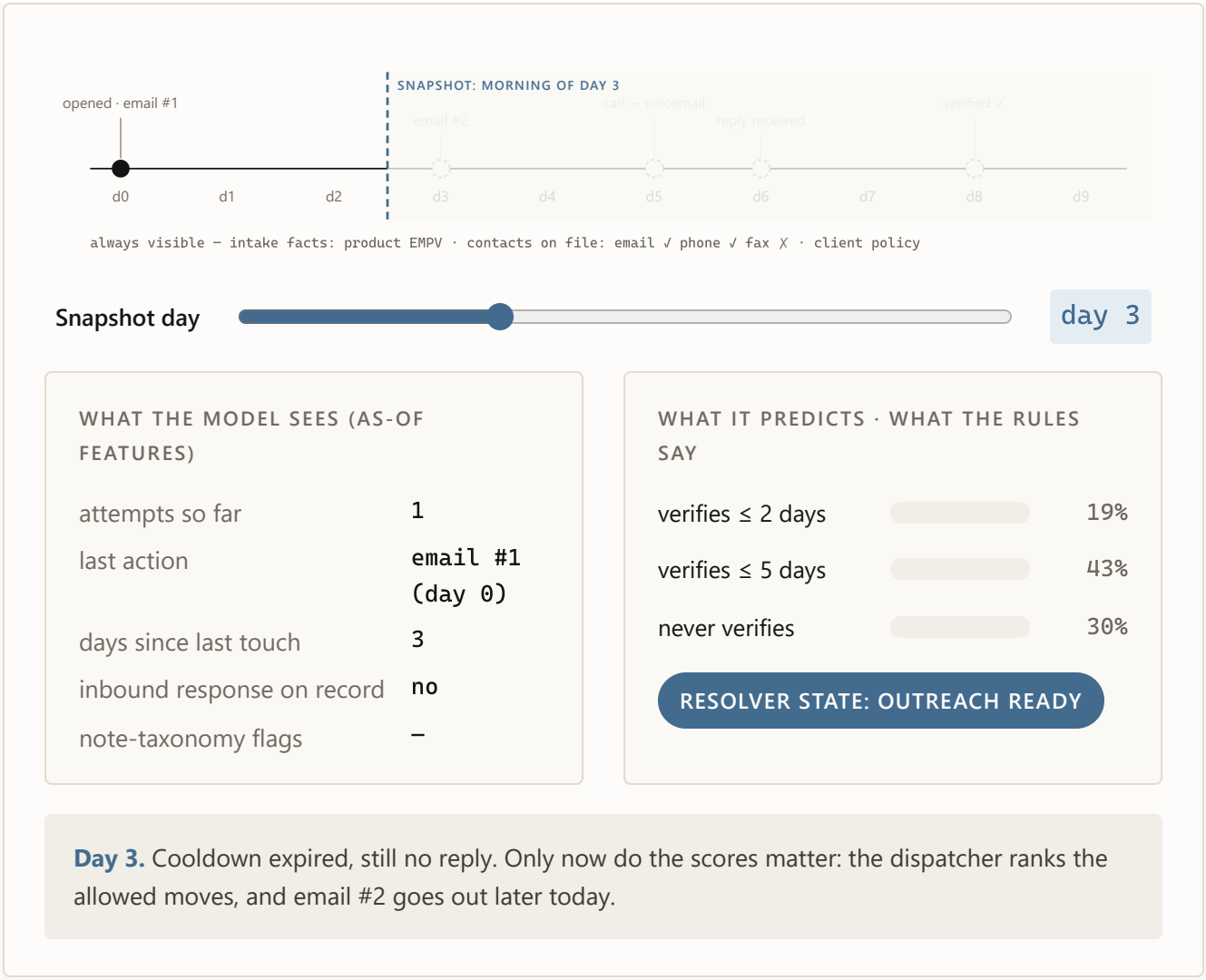


FIGURE 2 · EVIDENCE SHOWS The as-of curtain, interactively. Probabilities are illustrative. The curtain is enforced, not assumed: an automated audit re-checks it on every new training matrix before any model trains.

2 Triage: three questions about the same case

For case 4127, Monday morning — the day it arrives — is the sharpest read it will ever get, before the first attempt changes the evidence. Each morning after, three calibrated machine-learning models read the visible half of the file and score it: *will this verify within 2 days? within 5? will it ever?* Calibrated means the numbers behave like frequencies — across all cases scored "30%

never-verify," about 30% actually never verify. That property is what makes the scores safe to build queues and thresholds on.

One case's scores are mildly interesting. The population view is where triage earns its keep: sorted by predicted never-verify risk, the easiest tenth of cases fails 3% of the time and the hardest tenth 48%. Same process, same operators — a 16× spread in what was ever achievable. That spread powers the queue (work the winnable middle first) and a fairer scoreboard (measure misses against what was possible, not against a blended average).

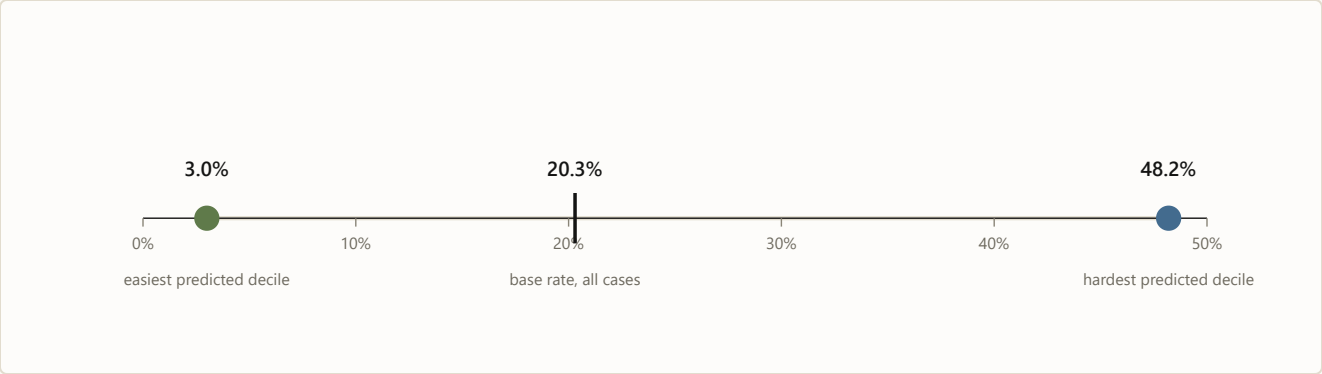


FIGURE 3 · ML PREDICTS Real numbers (May 2026 holdout): observed never-verify rate by predicted-risk decile. Triage doesn't change any single case; it changes which cases get the attention.

3 The checklist: what state is this case actually in?

By day 3, the live question about case 4127 is no longer how hard it is — it is what is true this morning: did anyone reply? is a cooldown running? is another touch allowed? Before anyone acts on a score, a deterministic resolver asks a fixed sequence of questions any good supervisor would ask — and the first "yes" decides the case's state for the day. The real resolver distinguishes thirteen states; the intuition fits in seven questions:

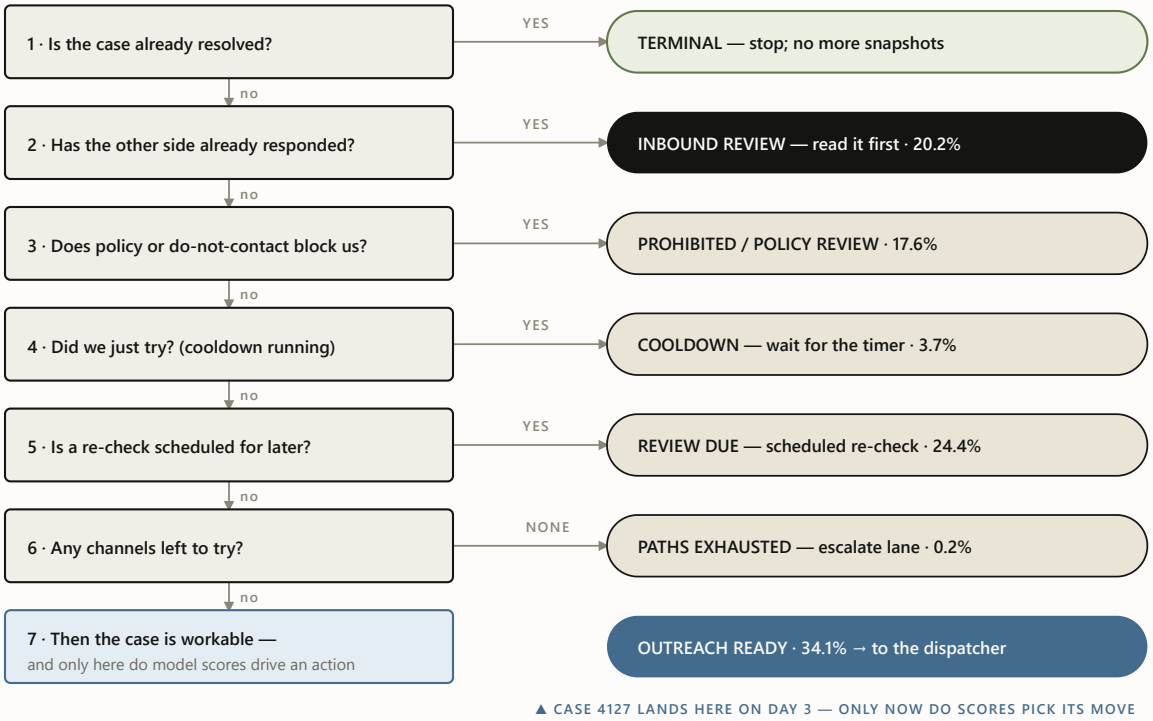


FIGURE 4 · RULES NAME The resolver as a supervisor's checklist (a seven-question simplification of the real thirteen states; percentages are each state's share of all 2.75M case-days). Notice question 2: a fifth of all case-days already have a response on record. Answering it deterministically is what caught the ~1-in-7 cases the old flow would have re-contacted. Across all 2.75M case-days: zero do-not-contact violations, zero policy violations — by construction, since scores are never consulted until row 7.

4 The bouncer and the dispatcher: six cards, one recommendation

Suppose it's day 3 and case 4127 landed on "outreach ready." The system now deals six possible moves — the same six for every case, 16.5 million candidate cards in the full run. Eligibility rules physically remove cards before any ranking happens; the model's opinion of a blocked card is irrelevant, because the card is no longer on the table.

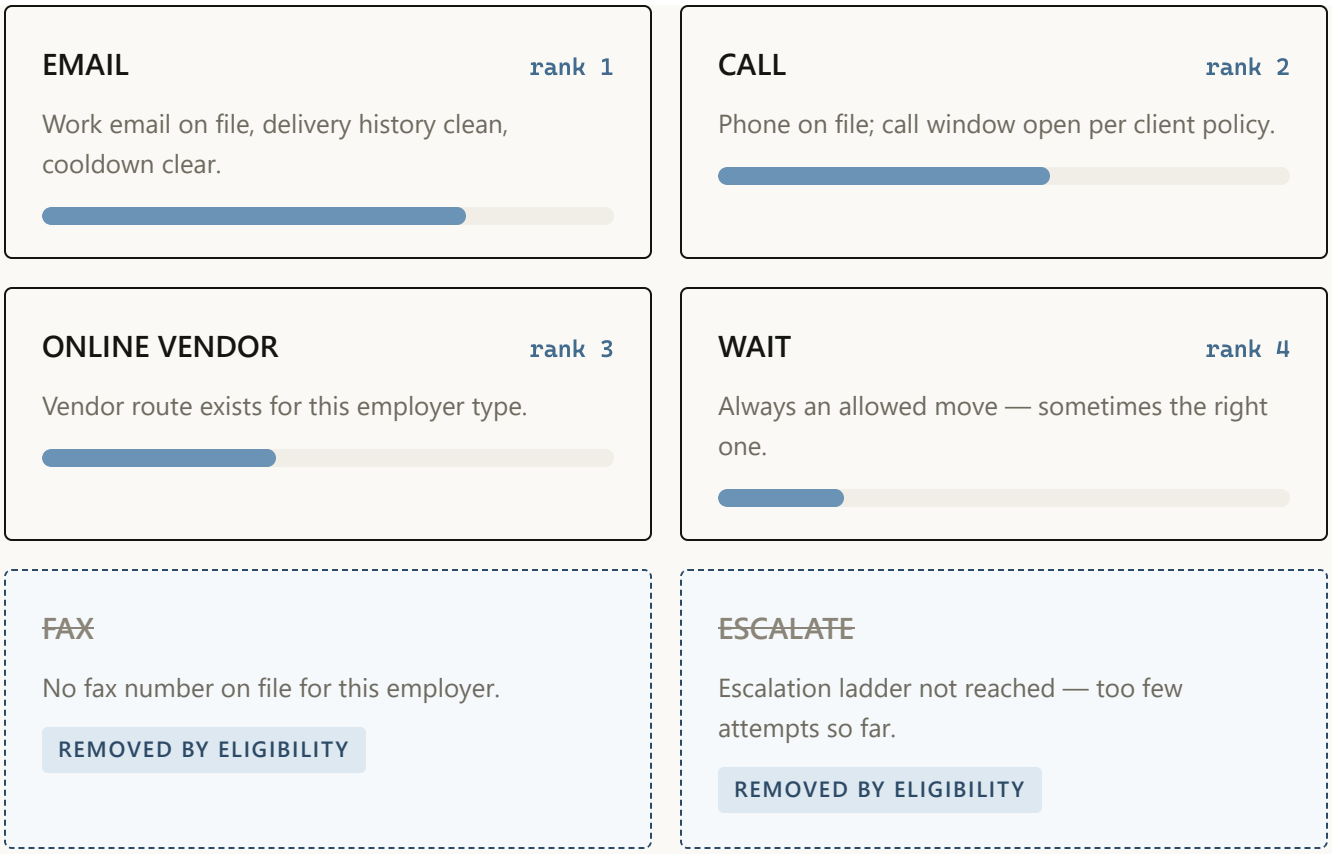


FIGURE 5 · RULES REMOVE, ML RANKS Case 4127's hand on day 3. Priority bars are illustrative; ranking reflects what historically moved similar cases under current policy — an observational signal, deliberately not a claim that any channel *causes* better outcomes. Blocked cards never reach the ranker: eligibility is a filter, not a penalty.

The dispatcher's output for the day is exactly one line: *recommended next move: EMAIL, rank 1 of 4 allowed*. In the full backtest, every single case-group ended with an available move — zero "rank-1 but blocked" contradictions, zero empty hands.

5 The flight recorder: a person decides, everything is written down

The recommendation lands in a queue in front of a person, who can take it or override it — and the override is as valuable a signal as the action. Five event types get logged, and together they form the case's complete decision story:

day 3 · 09:12	RECOMMENDED EMAIL — rank 1 of 4 allowed moves, shown to operator
day 3 · 09:14	ACTION EMAIL sent, no override · template family `follow_up` v3
day 4 · 07:40	DELIVERY delivered — no bounce
day 6 · 15:03	REPLY inbound response recorded → resolver flips to INBOUND REVIEW
day 8 · 11:26	OUTCOME verified — outcome joined back to every prior event

FIGURE 6 · PERSON DECIDES, LOG REMEMBERS Case 4127's flight record (fictional timestamps). This chain — recommendation, action or override, template version, delivery and reply, outcome — is the price of ever saying the word "because." Until it's collected at scale, the system reports correlations as observations and nothing more.

6 The heartbeat: how the machine stays honest month over month

Case 4127 verified on day 8. One case proves nothing — but its complete decision story joins tens of thousands of others in the next evaluation cycle. Everything above is the *daily* loop. Wrapped around it is a monthly one: new data lands, the snapshot matrix is rebuilt behind the curtain, an automated leakage audit checks the curtain held, challenger models are trained, and a mechanical gate decides — champion or challenger — with the verdict recorded either way. The gate has said no to three of my own improvements this year. That's the feature, not the bug.

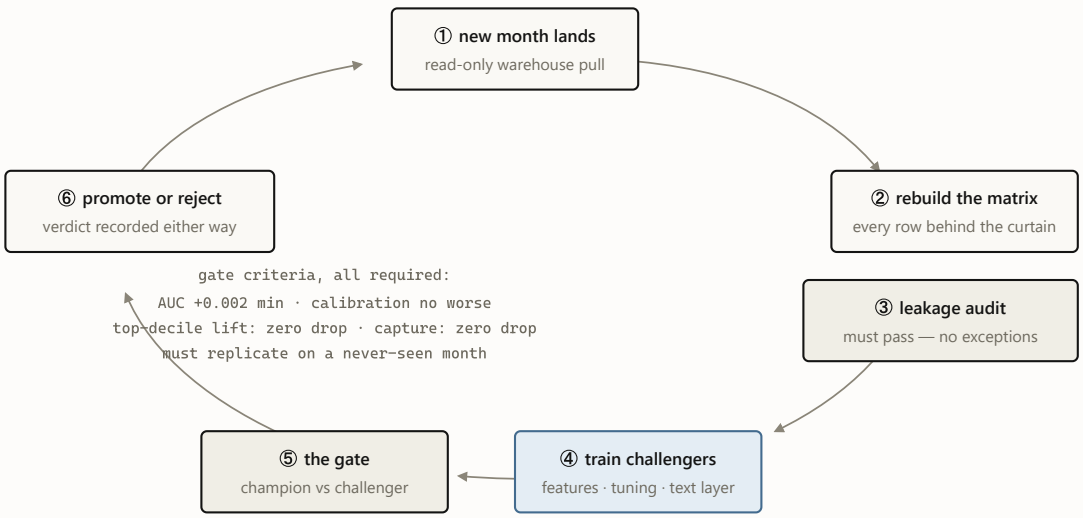


FIGURE 7 · OUTCOMES TEACH The monthly heartbeat. July 2026's cycle promoted text-aware champions for both success targets and rejected the same idea for the never-verify target — its third rejection this year, each one recorded in the champion manifest. A gate that never says no is a rubber stamp.

WHY TWO LANES AND NOT ONE BIG MODEL?

Because the two lanes fail differently. If triage is wrong, an operator works cases in a slightly worse order — cheap, recoverable, invisible. If "what's allowed" is wrong, someone gets contacted who must never be contacted — an incident with a name on it. Splitting the lanes means the expensive failure mode is handled by the component you can audit line by line, and the learning component is confined to the failure mode you can afford. A single end-to-end model would blend both failure modes into one — and price everything at the expensive one.

The whole machine today — and what case 4127 looks like next

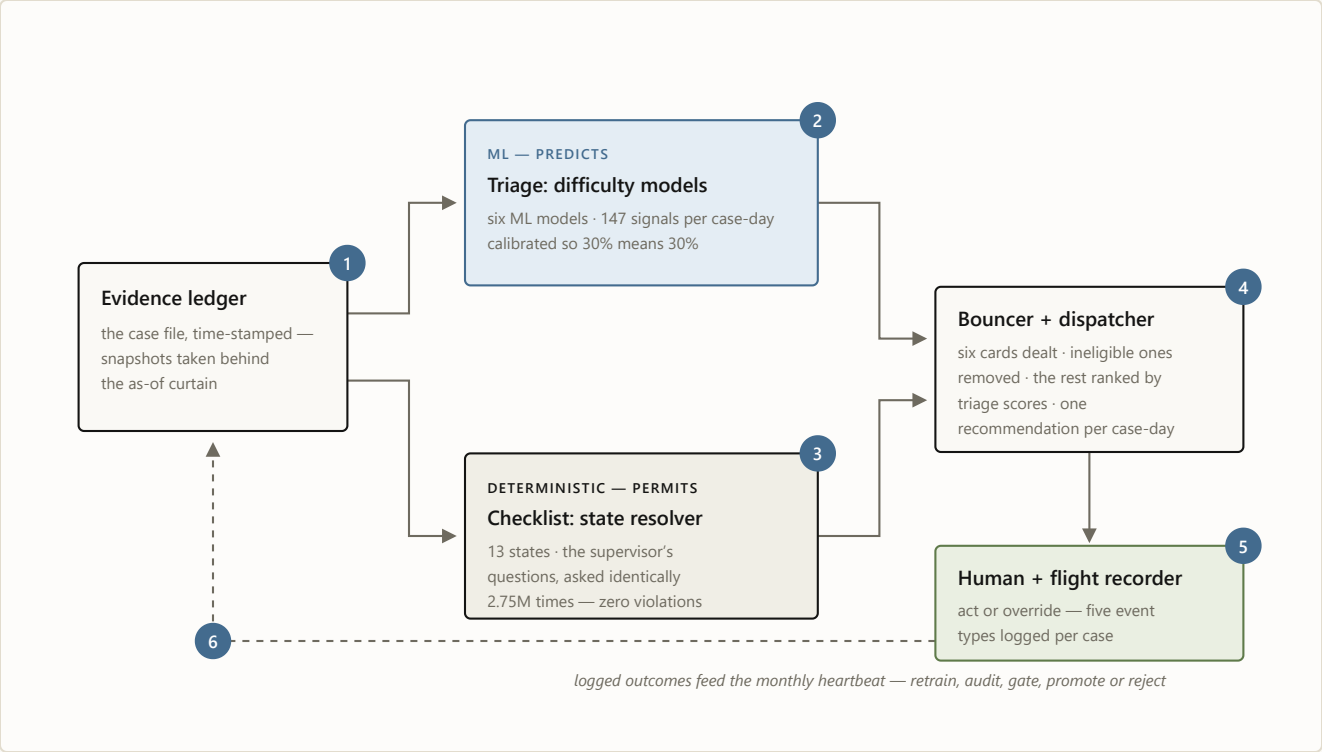


FIGURE 8 The assembled machine, with case 4127's six stops numbered in order: **1** the file is snapshotted behind the curtain · **2** triage scores it · **3** the checklist names its state · **4** the bouncer deals and the dispatcher ranks · **5** a person decides and the recorder writes it down · **6** the logs feed the monthly heartbeat. Blue predicts; oat permits; moss decides.

That's the whole machine, today. No step is mysterious on its own — a curtain, three scores, a checklist, a bouncer, a queue, a log — and that is deliberate. Each layer is simple enough to explain to the people whose work it touches, and the intelligence lives in how strictly the layers are allowed to talk to each other.

Now run the same case forward a stage. Today, the engine recognizes that case 4127 is ready for outreach, removes the blocked moves, and recommends email; a person reviews and sends it. In the next stage, the engine recommends the *complete play*: confirm the contact path is still good, email this morning

with the employment-confirmation message family, and schedule a call only if no reply lands inside the measured response window. If the path on file looks dead, an AI contact researcher hunts a fresh, verified one before another attempt is spent. A small language model drafts the email; a person approves or edits it, and the system records why. And if that complete play keeps winning across comparable cases, that one narrow lane can move to controlled auto-send — only after its logged record proves it safe, stable, and effective.

The case still passes through the same machine — the curtain, the checklist, and the bouncer never move. The difference is that the system gets better at filling the space inside them: state → allowed moves → complete play → path repaired if weak → draft prepared → a person approves, or an earned lane sends → outcome → the loop learns. Today the machine recommends the next move. The future machine learns the whole play — and every completed case makes the next one better. ◆

The series: Building a decision engine for verification casework (the engineering deep dive) · Earned autonomy (the design philosophy) · this illustrated guide · plus the executive briefing and the Murphy Brown field report.

Case 4127 is fictional and its probabilities, ranks, and timestamps are illustrative. Real numbers cited: 2,751,900 case-day snapshots; 16.5M candidate actions; state-mix percentages from the 2026-06-19 resolver run; the 3.0% / 48.2% decile spread from the May 2026 holdout; zero DNC and policy violations across the full backtest. Correlations are observational — no channel or wording is claimed to cause outcomes.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

A Wright Original · Built by **SNH Strategic Initiatives**

Every verification case ends one of two ways: verified, or *unable to verify*. In the twelve months ending May 2026, UBS worked nearly a million verification searches across four products — employment, education, reference, and license-style checks (EMPV, EDUV, REFV, PLV). Just under half a million of them received at least one attempt, and one in five of those — roughly 100,000 — ended unable to verify. That UTV rate is the number the business wants down, and it is a frustrating one to manage, because it blends two very

different populations: cases that were never going to verify no matter what anyone did, and cases that were winnable but stalled.

The system described here exists to separate those populations and act on the difference. For every open case on every day it answers two questions — *how hard is this case?* and *what is the next sensible move?* — and it answers them with a strict division of labor: machine-learned models predict, deterministic rules permit. This post explains how the pieces fit, what the evaluation discipline looks like, and the places the discipline said no. One expectation to set early: the largest immediately bankable finding in here did not come from a better model. It came from writing down, in rules, what "already responded" means.

Control changes hands five times

The architecture is five layers, and the arrows matter more than the boxes.

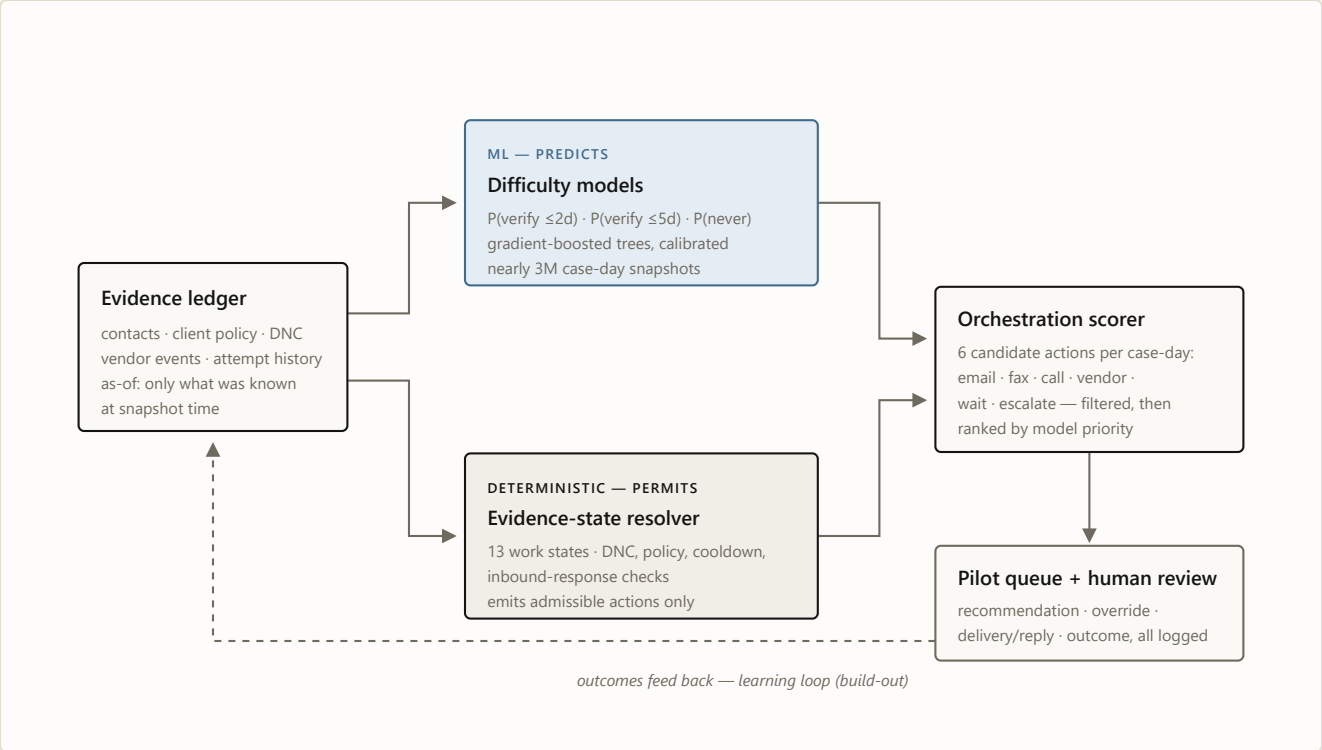


FIGURE 1 The decision path. Models and rules both read the same as-of evidence, but only the resolver decides what is allowed. The scorer ranks within allowed moves; a human works the queue. The dashed loop — learning channel, cadence, and message from logged outcomes — is the build-out, not the current system.

The load-bearing design rule is the one in the middle:

> *Deterministic rules decide what is allowed and what is blocking. Models only prioritize within allowed moves — a model never authorizes an action it cannot justify.*

Do-not-contact, client policy constraints, third-party exclusions, cooldown timers, and "this person already replied" are all resolved by rules, independent

of the models, before any score is consulted. A hard eligibility violation is a filter, not a score penalty a confident model could outvote. In the full backtest — nearly three million snapshots — the resolver produced **zero DNC violations and zero hard-policy violations**, which is only an interesting number because the models never got a chance to create one.

Three questions, one matrix

Three machine-learned models each take one version of "how hard is this case?" — will it verify within two days, within five days, and will it *ever* verify. All three are trained from a single matrix of case-day snapshots — one snapshot per open case per day, nearly three million over the twelve-month window — each restricted to what was knowable at that moment. The models themselves are gradient-boosted decision trees (the HistGradientBoosting family): an ensemble method that builds hundreds of small decision trees, each one trained to correct the mistakes of the trees before it. I chose that family deliberately over deep-learning alternatives — on structured operational data like this it is still the accuracy benchmark to beat; it natively handles the mix of counts, categories, and missing values that case history actually is; it retrains in minutes, which is what makes a monthly self-improvement cycle cheap; and its decisions can be interrogated feature by feature. On top of each model sits a calibration layer that turns raw scores into honest probabilities — when the system says 30%, it happens about 30% of the time — because the consumers of these scores are queues and thresholds, and a queue built on miscalibrated probabilities is a queue that lies about its own workload.

Table 1. Champion models (v3, promoted 2026-07-02). May 2026 is the holdout gate month — roughly 300,000 nev 2026 is a further out-of-time month scored with frozen bundles.

TARGET	CHAMPION VARIANT	BASE RATE	ROC-AUC, MAY	ROC-AUC, JUNE O
Never verifies (final UTV) the headline target	baseline, bounded features kept — 3 challengers rejected	0.201	0.706	champion held; lift 2.61× vs 2.4
Verifies within 5 days	note-aware, identity-safe promoted	0.477	0.717	0.7
Verifies within 2 days	note-aware promoted	0.264	0.747	0.7

In plain terms: hand the never-verify model any two open cases — one that eventually verified, one that never did — and it puts them in the right order **71% of the time**, where a coin flip manages 50% (that is what ROC-AUC 0.706 means; every score here reads the same way). The two-day and five-day models sort at 75% and 72%. At day 0 — before any work has happened — a walk-forward, identity-safe variant of the never-verify model reaches **≈81%** across three forward folds (82% / 80% / 81%); knowing on intake which cases probably won't come back is the single most useful thing the system does. The gap between 81% and the pooled 71% is survivorship, not decay: fresh cases differ enormously and are easy to tell apart, while the pool of still-open cases grows more alike as the easy ones resolve out — ranking the survivors is intrinsically harder. And the three champions are not alone: **six machine-learning models run in orchestration** — these three forecasts, the day-0 intake read, day-1/2 re-scores for aging cases, and an early-warning model — all reading the same morning snapshot, all feeding one daily decision pass.

What that buys operationally is the spread: the tenth of cases the model ranks riskiest actually fails **48% of the time against a department-wide base rate of 20%** — 2.4× the true never-verifies packed into the first list an operator works:

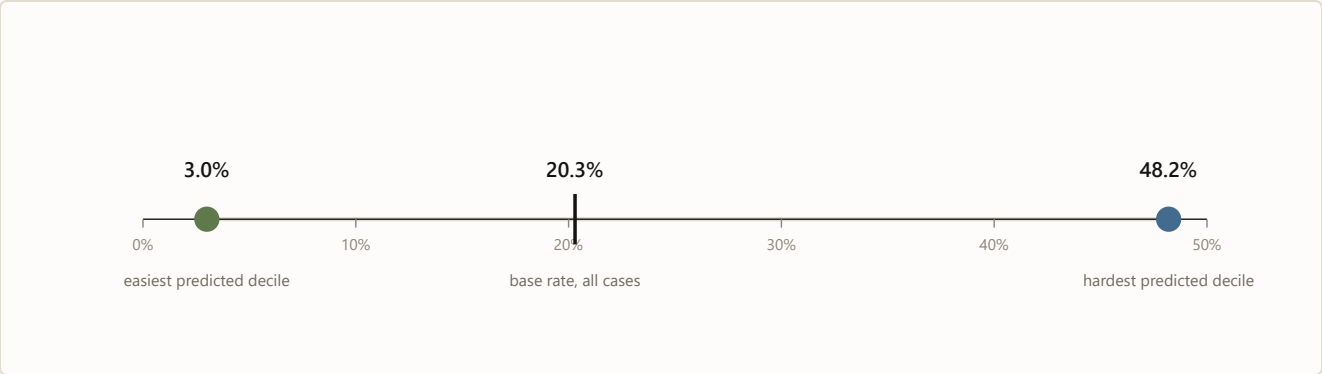


FIGURE 2 Observed unable-to-verify rate by predicted-risk decile, May 2026 holdout. The model separates cases that fail 3% of the time from cases that fail 48% of the time. That spread is what makes a *fair* UTV metric possible: misses can finally be measured against what was actually winnable.

That 16× spread is also the business story. Today's UTV rate silently blames operators for cases that were structurally dead on arrival. Score-conditioned targets flip that: the easy decile is expected to verify almost always, the hard decile almost never, and management attention goes to the winnable middle.

The three champions are also just the tip of the fleet. Six machine-learning models run in this system's production posture, each with its own validation story:

Table 2. The model fleet. All are gradient-boosted trees, calibrated so their probabilities read true (a 30% means 30%), trained under the same as-of matrix discipline; every model passes the leakage audit, and champions additionally clear the promotion gate.

MODEL	PREDICTS	ROC-AUC	VALIDATION	STATUS
Never-verify champion	will the case ever verify	0.706	May holdout + June OOT	champion — bounded

MODEL	PREDICTS	ROC-AUC	VALIDATION	STATUS
Verify within 2 days	resolves ≤2d of snapshot	0.747	May holdout + June OOT	champion — text features
Verify within 5 days	resolves ≤5d of snapshot	0.717	May holdout + June OOT	champion — text, identity-safe
Day-0 intake read	never-verify, at arrival	~0.81	walk-forward, 3 forward months	identity-safe variant
Day-1/2 re-score	re-reads aging cases	0.73 / 0.70	holdout	lifts aging EMPV read 0.62 → 0.71
Early warning	will strand by day 20	0.776	single-month holdout	preliminary

A high score still cannot break a rule

The models answer “what is likely?” The resolver answers “what is true, what is blocked, and what may happen next?” The questions stay separate because they fail differently: a bad ranking wastes effort; a bad permission creates an incident. The deterministic half is an evidence-state resolver that classifies every case-day into one of thirteen work states — terminal, already-responded, policy-blocked, awaiting response, cooldown, review-due, research-needed, digital-paths-exhausted, standard-paths-exhausted, outreach-ready, and so on — and derives the set of admissible actions from the state, never from a score. Here is where the 2.75M snapshots actually sit:

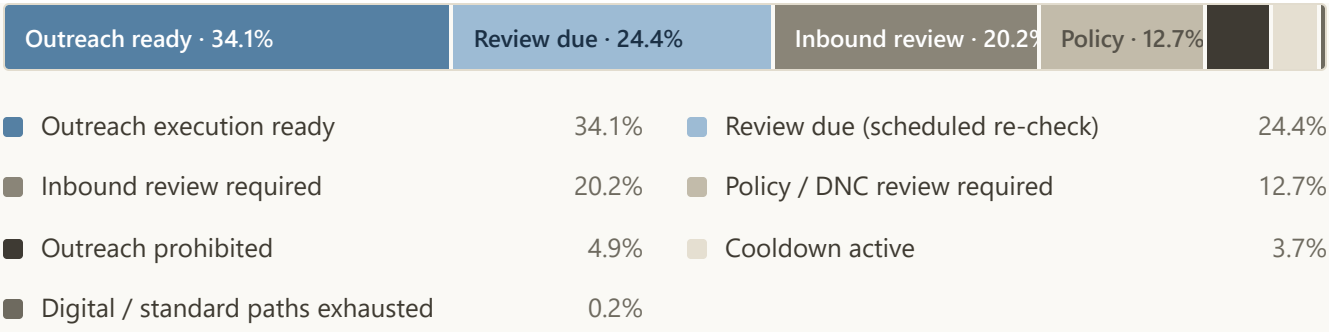


FIGURE 3 Resolver state mix across nearly three million snapshots. The interesting segment is the third one: cases where a vendor response is already on record but the legacy flow would have kept contacting.

I expected the next win here to come from a better model. It came from a rule — and it was hiding in that third segment. On roughly **400,000 case-days — about one in seven** — a vendor-channel response had already been received, yet the process as practiced would have actively re-contacted the party. The resolver routes every one of them to review-first instead. No model was needed for this; it fell out of writing down, deterministically, what "already responded" means. (The signal is a vendor receipt, not parsed reply content, and it doesn't exist for every product — the honest version of this number lives in the repo docs alongside its caveats.)

The same rule also shows why the safety claim holds under pressure. Suppose a model strongly favors another call on one of those cases. The resolver sees that a reply is already on record, so call is removed before the final ranking — the model's confidence cannot restore it, and the person never even sees it as a recommendation. That is why the replay produced zero policy violations: safety does not depend on the model remembering every rule. The model never receives the authority to break one.

Downstream of the resolver, the scorer builds six candidate actions per case-day — email, fax, call, online vendor, wait, escalate — 16.5 million candidate

actions in the full run, filters them through eligibility, and ranks the survivors by the calibrated scores. A seven-stage pipeline chains candidates → eligibility → scoring → call-escalation ladder → wait policy → workflow router → pilot queue; every stage checks its own output and halts the run on any failure, and a strict contract validator signs off at the end. The full run produced zero blocked rank-1 actions, zero no-action groups, and a queue where every group has an available move.

One case-day, from evidence to action

Here is the whole handoff on a single day, using the illustrated guide's fictional case 4127. On day three, the case enters as a morning snapshot. The machine-learning models estimate its likely path. The resolver names it outreach-ready. The rules remove fax, because no fax number exists, and remove escalation, because the case hasn't reached that rung. The scorer ranks the four surviving moves and places email first. A person approves the recommendation. The log records the recommendation, the decision, the message, the reply, and the final outcome.

The models never see six equally available actions. The operating system first decides which actions are real.

No peeking, by construction

The easiest way for a model like this to look brilliant is to let information from after the outcome leak into its features — post-outcome status fields, completion timestamps, identity proxies. Every feature here is built under

strict “as-of” discipline instead: 117 columns, each computed with strictly-before semantics, so a case scored on a Tuesday morning is scored only on what was knowable that Tuesday morning. An automated leakage audit re-checks that boundary on every new training matrix before any model is allowed to train on it — and runs again at every monthly retrain. The scores in this post are conservative by design, measured the hard, honest way.

The system rejected three of my improvements

Model improvements are promoted per target — there is no single “champion model,” there are three — and promotion is mechanical. A challenger must clear every criterion against the sitting incumbent; a single checked-in champion config is the source of truth, and rejections are machine-recorded in the champion manifest.

ROC-AUC	at least +0.002 over the incumbent, with bootstrap confidence intervals excluding zero
Brier score	no worse than the incumbent — calibration is not negotiable
Top-decile lift	zero drop tolerated — the top of the queue is what operators actually work
Capture at top 10%	zero drop tolerated
Slice gate	per-product and per-day slices, zero-tolerance with a reviewed, unit-tested allowlist for false positives
Out-of-time replication	the win must survive a never-seen month scored with frozen bundles

The gate earns its keep on what it rejects. Three times this year I tried to improve the never-verify model — a settings-tuning pass, a guarded version of the same sweep, and this cycle's note-reading variant — and the gate turned all three away **for the same reason**: each won on average, and lost where operators actually work — the top of the ranked worklist, the first page, where accuracy is worth the most. One rejection is bad luck; three identical ones is a message. For this target, average accuracy and top-of-list sharpness genuinely pull against each other, so the next attempt changes the training data itself — June's outcomes finish maturing in the July pull — rather than pushing a fourth model variant.

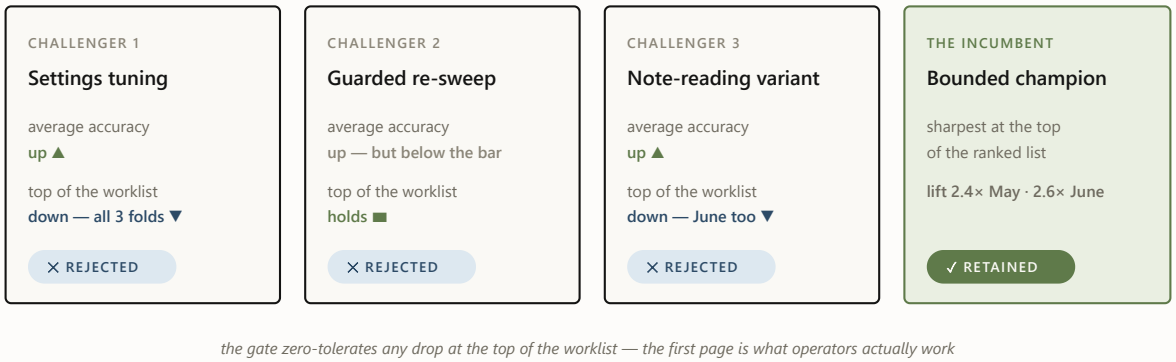


FIGURE 4 The gate at work on the never-verify target: three challengers, three different levers, one identical failure mode. Each verdict is machine-recorded — the rejections are as documented as the promotions.

A gate that never says no is a rubber stamp. Ours says no to us regularly, in writing.

Reading the notes without keeping them

The biggest accuracy gain this cycle came from the data source I had been most careful to avoid: the free-text notes operators and vendors leave on case history (“left a voicemail”; “reached HR, call back Tuesday”). Notes are where the richest signal lives — and where the risk lives twice over: outcome language (“verified via…”) leaks answers into training, and raw text carries the privacy exposure, so persisting it into training artifacts was never on the table.

The design that threads the needle: **a deterministic reading layer classifies each note the moment it streams past — then throws the text away**. Every note is sorted into eleven kinds of operational signal (contact made, voicemail left, callback promised, redirected to a third party, …), and only the signal survives: which kinds of events a case has seen, how recently, how often. Deterministic matters here — the same note always produces the same labels, so the privacy boundary has no black box in it and the whole layer can be audited and replayed. That turns 17.9 million unreadable notes into 30 new machine-usable signals per case-day for the learned models, computed under the same no-peeking time discipline as everything else; about 28% of case-days carry at least one.

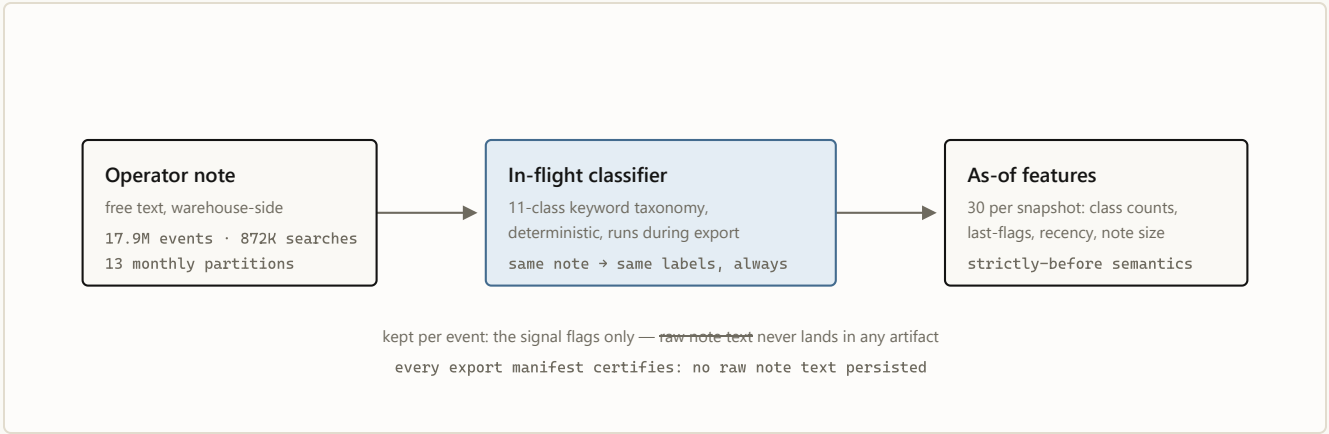


FIGURE 5 The inbound text layer. Signal crosses the boundary; text does not.

The same new evidence helped two forecasts and hurt the third. The payoff cleared the full promotion gate for both success targets — the five-day model improved on every metric with confidence intervals excluding zero, and the gain *grew* on the never-seen month; the two-day model held the same pattern. The never-verify variant, as covered above, gained on average, lost the top decile, and died at the gate.

One operational note survives from that build: a routine row-count check against the export manifests caught a corrupted export *before anything consumed it*. Cheap invariant checks between pipeline stages are the highest-ROI code in the system.

What the evidence still will not let me say

The layers ahead are ambitious, which makes the boundary between built, ready, and not-yet-proved more important — not less. The repo maintains an explicit disallowed-claims list, and it does more for the project's credibility than any AUC. The current lines:

- **No causal claims about channel, wording, or templates.** Historical patterns like "email-first cases verified more often than call-first cases" are observational — selection effects all the way down. Causal language stays banned until the pilot logs the full chain for each case: recommendation shown, action taken or override reason, template version, delivery/reply, final outcome. The logging schema for exactly those five event types is built and validated; the evidence is not yet collected.
- **Not "identity-free," only "direct identifiers stripped."** Action-facing models exclude direct agent and team IDs behind a drift gate, but

behavioral agent attributes remain. Identity-bearing champions stay `provisional` for action-facing use, and the resolver applies no-direct-identity fallbacks automatically until that gate approves.

- **Decision support today — automation is earned, not assumed.** Nothing auto-sends yet and no decision is finalized by a machine; the wiring for automation exists, and each step toward it — exploration, auto-drafts, then gated auto-send — unlocks on pilot evidence, with exploration shipping disabled by default. The router is shadow-review metadata, not an autonomous policy — and I won't train a "learned policy" on historical action logs and pretend the logs were experiments.
- **The language layer is next-phase, on purpose.** The endgame — a small language model that drafts the emails, faxes, notes, and call scripts, learning from past communication to draft the best next action — is sequenced last because without the causal instrumentation above no one could tell whether it works.
- **No rework-savings claims — the QA loop is measured, not yet moved.** A reproducible July baseline timed the quality-review loop (education + employment, Jan–Jun 2026): a returned check runs a median 11.2 days from first QA return to approval, against 5.0 days for a typical check's entire first pass — and 85.6% of coded return reasons are process-and-judgment classes, the ones the resolver, eligibility rules and lanes exist to prevent. "Exist to prevent" is a design statement, not an effect size: the effect claim waits for the pilot, and hands-on handle time waits for work-item state logs that nothing emits today.

What the live record teaches next

The engine today solves three foundational problems: it reads the case, decides what is allowed, and recommends the next move. Each layer that follows expands what a “move” contains — and each becomes possible because the one below it already exists. The resolver supplies the state; the rules supply the boundaries; the pilot supplies the evidence. Concretely: the recommendation is logged; the person accepts or overrides it; the chosen channel, timing, message, delivery, reply, and final outcome are recorded. That complete chain — not the final outcome alone — is what teaches the system which play worked.

Three threads, in order of readiness. The **July cycle** re-runs the monthly spine — the text exporter and matrix builders are month-partitioned and resumable for exactly this — and matures June’s never-verify labels, which is the most promising lever left for the one target that keeps rejecting challengers. The **remaining pre-integration checks** either approve the identity-bearing champions for action-facing use or keep the fallbacks in place; the walk-forward drift harness exists, and the decision is deliberately not mine to hand-wave. And the **play engine** — recommending channel + timing + message family + template as a unit, then eventually learning that choice from pilot data that records which options were offered, not just which was taken — waits on the only thing that can unlock it: real logged outcomes from humans working the queue.

One more build sits on the same roadmap: a **contact-researcher loop** — an AI agent that goes looking for better contact paths (fresh, verified emails, fax numbers, and phone numbers for employers, registrars, and vendors) and hands them to the eligibility layer as an upgraded contact plan. Today a case

lives or dies on the contacts already in its file; a researcher that replaces a dead fax line before the third failed attempt turns unworkable cases workable. Like every other layer, its suggestions enter as candidates for the rules to admit — never as automatic dial targets.

And after the play engine comes the final step: a **small language model that writes the communications themselves** — the emails, the faxes, the case notes, and the scripts our call operators work from — learning from the logged record of past communication and its outcomes to draft the best next action for every case. It is sequenced last because it is graded last: only the pilot's causal instrumentation can tell a well-worded email from a lucky one.

And behind all of it sits the layer every vendor sells first: execution. Every lane starts with human review. A narrow, stable lane moves to controlled auto-send only after its logged evidence proves it safe, stable, and effective — authority expands one lane at a time, from measured outcomes, never from a roadmap date.

The system today is, on purpose, the boring version: calibrated probabilities, deterministic guardrails, mechanical promotion gates, and a written list of things I refuse to say. Every part of it is replaceable — the models get retrained, the rules will evolve, the contact researcher and the language model still have to earn their place. What persists is the handoff: evidence enters, machine learning predicts, rules permit, a person decides, and the outcome becomes the next case's evidence. The models provide the intelligence; the operating system turns it into work the operation can trust. ◆

The series. This is the engineering deep dive. Also: Earned autonomy (the design philosophy) · the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

There is a class of problem that looks made for AI and punishes naive AI harder than almost anything else. Call it *operational casework*: hundreds of thousands of open cases, each needing a next move today; hard compliance constraints — do-not-contact lists, client policy, regulated channels; human operators who own the outcomes; and labels that arrive late, noisy, and entangled with the process that produced them. Verification work is our instance — roughly 880,000 cases in a year, a fifth of the attempted ones ending “unable to verify” — but collections, claims, case management, and outreach operations all share the shape.

The tempting move is the end-to-end agent: feed it the case, let it decide, act, and learn. In this problem class that design fails in a specific, predictable order. First it violates a constraint, because a model treats policy as a feature with a weight rather than a boundary — and at our volume, a 99.9%-compliant model is thousands of violations. Then it flatters itself, because historical logs reward whatever the process already did. And then it can't be trusted with the interesting decisions, because nothing in its training data identifies what would have happened under a different action. Our own verification bot is the live case study: it runs without this operating layer, and the field report on what it leaves behind — ungated pickup, note-only handoffs, reopening closures — is this essay's companion piece.

So I built the opposite of an end-to-end agent — because in this class of work, the fastest route to useful autonomy begins by refusing it. The interesting part isn't the system; it's the discipline. Four rules, in increasing order of how much they cost us.

1. Split the decision

Every "what should we do next?" question decomposes into two questions with different failure tolerances. *What is permitted?* has a tolerance of zero: getting it wrong is an incident, and the answer must be explainable after the fact, case by case. *What is preferred?* has a tolerance of "be usefully better than the status quo": getting it wrong wastes effort. The pattern's first rule is to never let one component answer both.

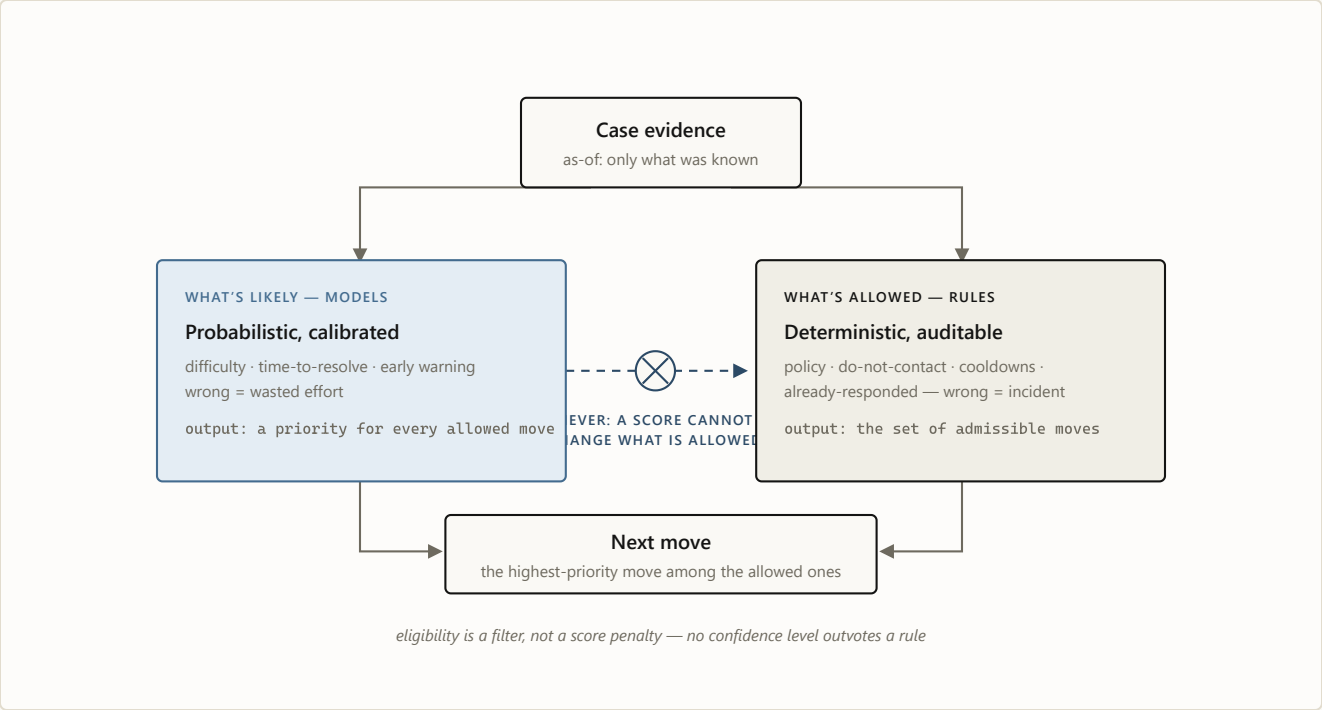


FIGURE 1 The split. Both lanes read the same evidence, but only the deterministic lane can say no. In this system the rules lane resolved 2.75 million case-days into thirteen work states with zero do-not-contact and zero hard-policy violations — an unremarkable number, which is the point: the models never had the opportunity to create one.

The split also produced the cheapest large win in the project, and it came from the *rules* lane. Writing down, deterministically, what "this person already responded" means revealed that on roughly one case-day in seven, a response was already on record while the process as practiced would have kept contacting. No model, no training run — just a definition, applied consistently at scale.

2. Forecast first, act later

The second rule is about sequencing: the first machine-learned product should be a *forecast*, not an action. Forecasts pay rent under zero autonomy. A calibrated estimate of "this case will never verify" is immediately useful for

triage — work the winnable middle, not the doomed tail — for early warning, and for something subtler that turned out to matter most to the business: **fixing the metric itself**.

An aggregate failure rate blames operators for cases that were structurally dead on arrival. A difficulty model unblinds that: the easiest predicted decile fails 3% of the time, the hardest 48%. Once you can condition on difficulty, "how are we doing?" becomes "how are we doing *against what was achievable?*" — a fair scoreboard, and a fundamentally different management conversation. The model changed what the organization measures before it changed anything the organization does.

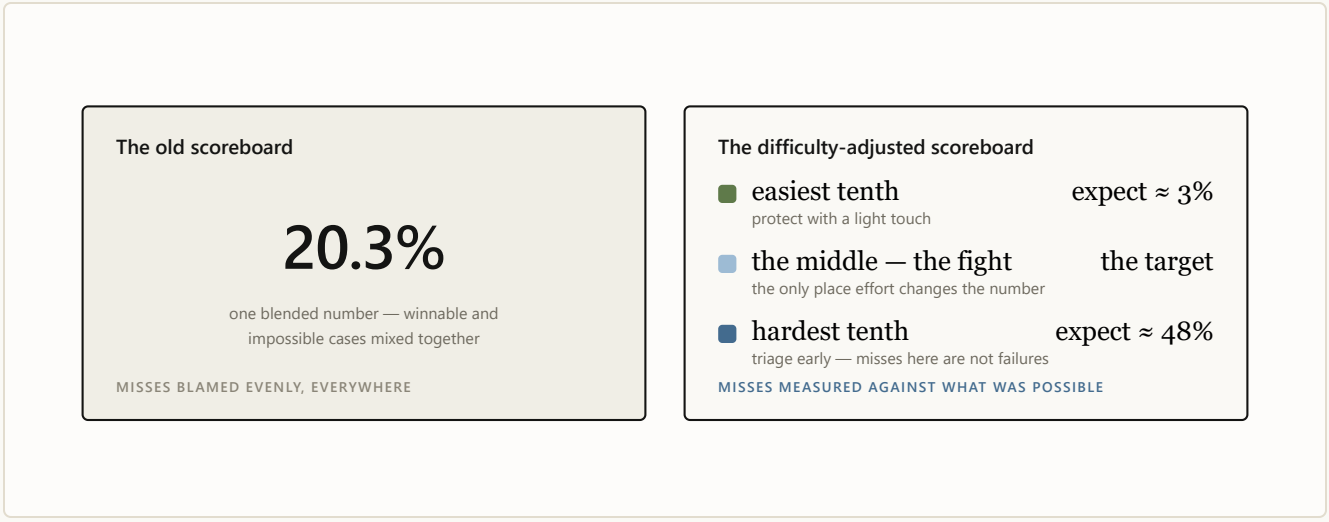


FIGURE 2 Same cases, two scoreboards. The blended number can only go up or down; the difficulty-adjusted one says where to spend effort and whom to stop blaming. This reframe — not any single prediction — is the business product of the forecast layer.

Forecast-first has a quiet second benefit: it forces the data discipline that everything later depends on — strictly as-of features, automated leakage audits, calibration checks — while the stakes are still just a number on a validation report. If the first product were an acting agent instead of a forecast,

a data problem would arrive as behavior in production; in a forecast layer it arrives as a number you can catch and fix before anything acts on it.

Restraint, to be clear, is not absence. Inside this pattern run six learned models over a 147-signal read of every case-day, retrained monthly under audit. The discipline is not a substitute for the machine learning — it is what makes that much machine learning safe to run.

3. Price every claim

Most ML systems fail socially before they fail technically: they say more than they know, someone believes it, and the credibility never recovers. The third rule is to treat claims as things with a price — and to refuse to make any claim whose instrumentation you haven't built.

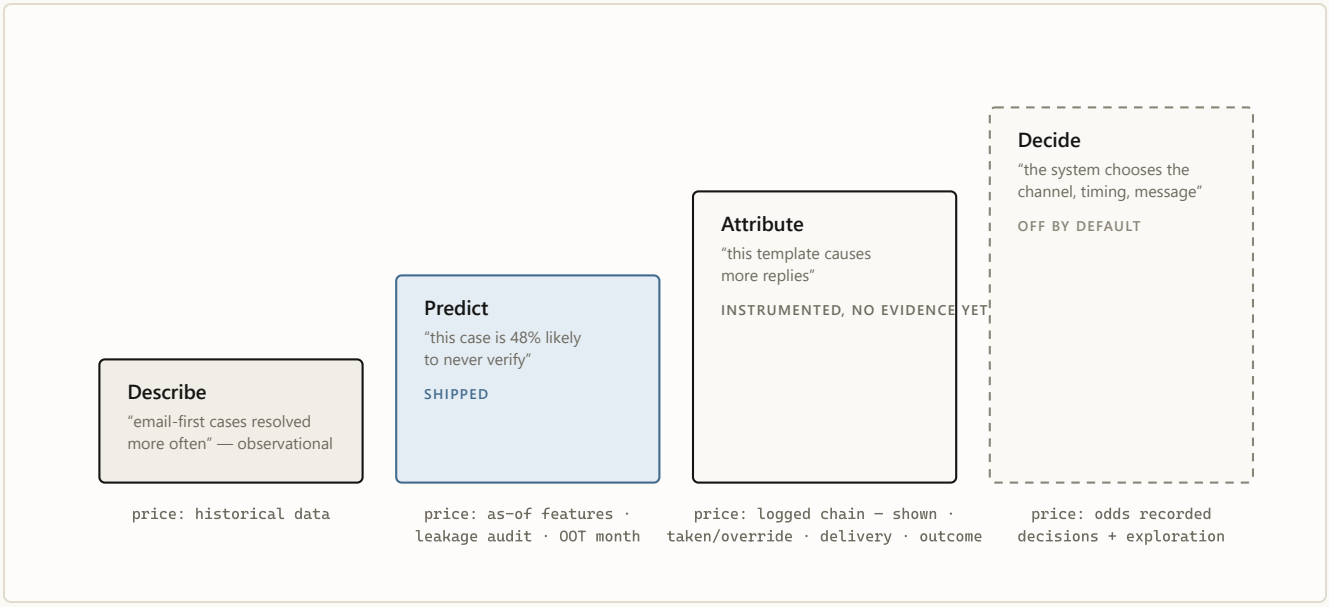


FIGURE 3 The ladder of claims. Each rung is priced in instrumentation, and the price is paid before the claim is made, not after. I hold the line publicly at rung two: correlations like "email-first resolves more often" are reported as observation, never as advice, because selection effects — not wording — may be doing all the work.

Two artifacts make this rule real rather than aspirational. The first is a written **disallowed-claims list** in the architecture record — no causal channel or template claims, no calling the router an autonomous policy, no training a "learned policy" on historical logs and pretending the logs were experiments. The second is a **mechanical promotion gate** for the models themselves: a challenger must beat the champion on ranking quality without giving up any top-of-queue sharpness or calibration, with confidence intervals excluding zero, replicated on a never-seen month. That gate has rejected three consecutive improvements to the headline model — each better on average, each worse where operators actually work. Three rejections in writing is the system functioning, not failing.

Autonomy is not a capability you install. It is a permission the evidence grants — and the default answer is no.

---

4. Let the system earn its next layer

The final rule turns the other three into a trajectory. Authority is granted in stages, each stage gated on evidence the previous stage was designed to produce — and every gate defaults to closed.

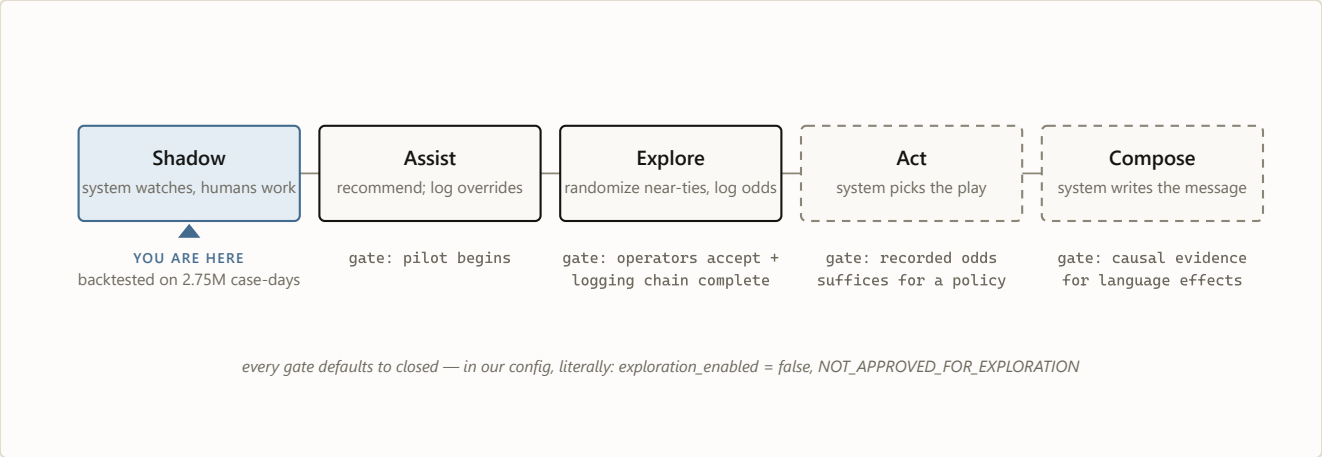


FIGURE 4 The autonomy ladder. Each stage exists to generate the evidence the next stage needs: the pilot logs overrides, overrides calibrate trust, exploration records each option's odds of being offered, and those recorded odds make a learned policy honest. Skipping a rung doesn't accelerate the system — it caps how far it can ever be trusted to go.

This is why the system's most capable component is the one that doesn't exist yet — and that's the ladder working as intended, not a gap in it. The most capable layers are the least verifiable ones, so their evidence requirements sit at the very top: composing outreach is the last rung exactly because nothing below the fourth rung can grade it. The roadmap names the destination; the discipline decides the pace.

What restraint buys

The pattern has real costs, and it pays for them. The ledger, honestly kept:

WHAT IT COSTS	WHAT IT BUYS
Slower headline progress — the gate rejects your own improvements, and rejections don't make launch announcements.	Deployability from day one — a system that provably cannot violate policy can enter a regulated workflow while still learning to be useful.
Boring demos — "calibrated forecast plus rules" never wows a room.	Failures that are cheap and legible — a wrong model means a suboptimal ordering, not an incident with a name on it.
Unglamorous work first — logging schemas, leakage audits, and manifests before any intelligence.	A metric the business can act on — difficulty-adjusted targets outlive any particular model version.
Self-correction on the record — every gate rejection and validation miss is written down, forever.	Credibility that compounds — every recorded "no" is what makes the eventual "yes" believable.

FIGURE 5 The trade, side by side. Everything on the left is real and annoying; everything on the right is why the pattern survives contact with a regulated business.

WHEN NOT TO USE THIS PATTERN

The full apparatus is overkill where its costs buy nothing: actions that are cheap and reversible, no compliance surface, outcomes observable within minutes, no human in the loop to protect. Ranking a newsletter's subject lines needs an A/B test, not an evidence-state resolver. The pattern earns its weight precisely where mistakes are expensive, constraints are regulatory rather than aesthetic, and trust — of operators and regulators — is the scarce resource.

The boring version, on purpose

One level down, this system is pipelines, gates, and calibration curves — the companion post walks through all of it. At this level it is a single idea applied four ways: **separate the question of permission from the question of preference, and make every expansion of the system's authority something the evidence, not the roadmap, decides.** Today that discipline produces forecasts, recommendations, and human review. Tomorrow the same discipline carries the rest of the destination: the complete play learned from logged outcomes, an AI researcher that repairs dead contact paths before attempts are wasted, a small language model that drafts the outreach — and narrow lanes moving to controlled auto-send only after the logged evidence proves them safe, stable, and effective. The path is ambitious *because* the gates are real. Restraint is not the absence of progress; it is what makes progress safe to grant. Restraint is the launch mechanism, not the destination. ◆

The series. This essay is the design philosophy. Also: the engineering deep dive (architecture, champions, the promotion gate) · the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

Internal note written in the style of a public post. All figures are backtested and as-of; correlations are reported as observation, never as causal advice; pilot results are not claimed.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

AFTER THE FIVE PIECES

The operation we are building

Six machine-learning models already read every open case and predict its path; deterministic rules remove unsafe actions; the engine recommends the next move. That claim rests on 2.75 million replayed case-days and zero policy violations — not on a destination slide.

The live workflow records what the system recommended, what a person chose, what was sent, who replied, and how the case ended. That record teaches the machine-learning models which complete plays work, sends an AI researcher after better contact paths, and gives a small language model the evidence to draft the outreach. A narrow lane moves to controlled auto-send only after its logged results prove it safe, stable, and effective.

The first release recommends. The destination learns.

The engine begins by recommending the next move. It ends as an operation that learns how to complete the case.